

Project Overview & Implementation

This project involves:

1. **Data Architecture:** Designing a Modern Data Warehouse Using Medallion Architecture **Bronze, Silver,** and **Gold** layers.
 2. **ETL Pipelines:** Extracting, transforming, and loading data from source systems into the warehouse.
 3. **Data Modeling:** Developing fact and dimension tables optimized for analytical queries.
 4. **Analytics & Reporting:** Creating SQL-based reports and dashboards for actionable insights.
-

In this Data Warehouse Project I will be using the Following ETL Methods

Extraction

- We will be using **Pull Extraction** as extraction Method
- For Extraction Type , we will be using **Full Extraction**
- For Extraction Technique , we will be using **File Parsing**

Transformations

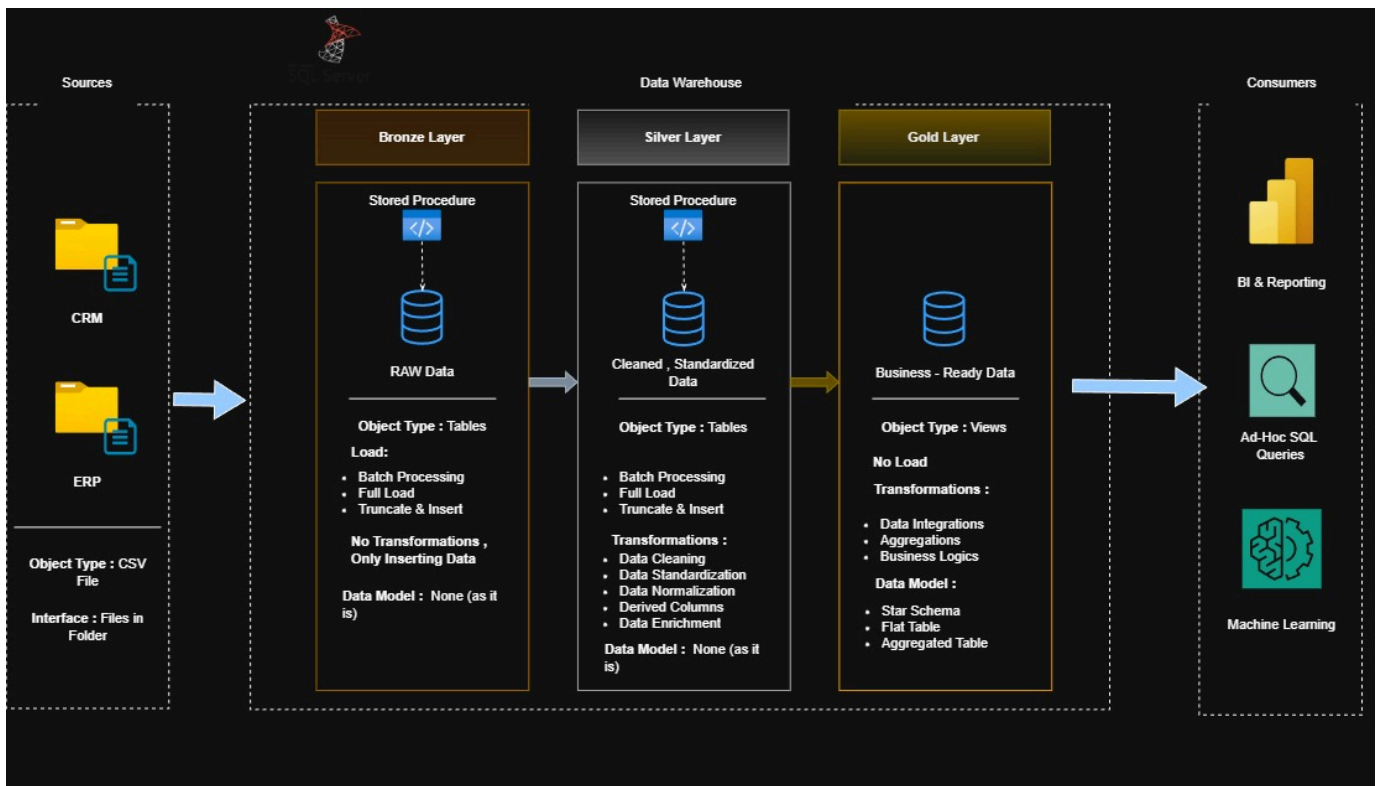
- All the Types of Transformations
 - Data enrichment
 - Data integration
 - Derived Columns
 - Data Normalization & Standardization
 - Business Logics
 - Data Aggregations
 - All the Types of Data Cleaning

Load

- For Processing Type , we will be using **Batch Processing**
 - For load methods , we will be using **Full Load --> Truncate & Insert**
 - For Historizations (SCD) , we will be doing **SCD 1 (Overwrite)**
-

Data Architecture

The data architecture for this project follows **Medallion Architecture Bronze, Silver, and Gold** layers:

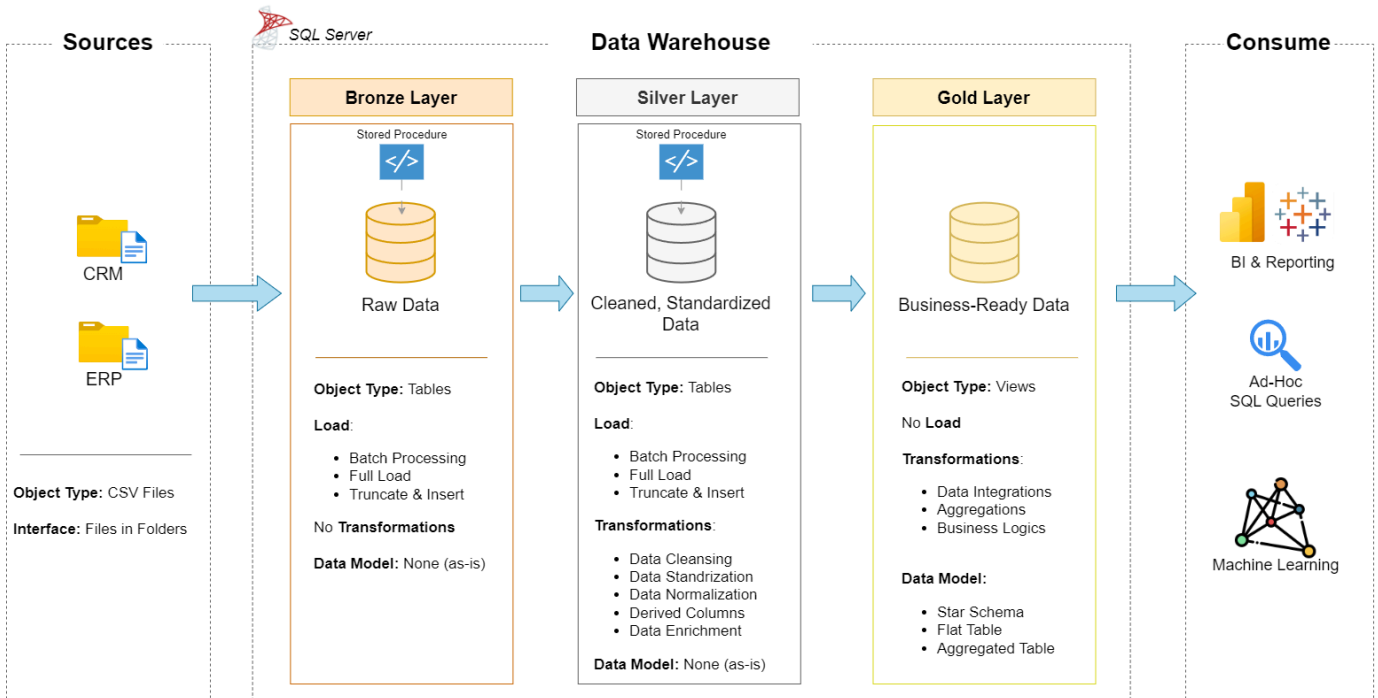


High Level Architecture

DATA WITH BARAA

in www YouTube

Baraa



	 Bronze Layer	 Silver Layer	 Gold Layer
Definition	Raw, unprocessed data as-is from sources	Clean & standardized data	Business-Ready data
Objective	Traceability & Debugging	(Intermediate Layer) Prepare Data for Analysis	Provide data to be consumed for reporting & Analytics
Object Type	Tables	Tables	Views
Load Method	Full Load (Truncate & Insert)	Full Load (Truncate & Insert)	None
Data Transformation	None (as-is)	- Data Cleaning - Data Standardization - Data Normalization - Derived Columns - Data Enrichment	- Data Integration - Data Aggregation - Business Logic & Rules
Data Modeling	None (as-is)	None (as-is)	- Start Schema - Aggregated Objects - Flat Tables
Target Audience	- Data Engineers	- Data Analysts - Data Engineers	- Data Analysts - Business Users 

1. **Bronze Layer:** Stores raw data as-is from the source systems. Data is ingested from CSV Files into SQL Server Database.
2. **Silver Layer:** This layer includes data cleansing, standardization, and normalization processes to prepare data for analysis.
3. **Gold Layer:** Houses business-ready data modeled into a star schema required for reporting and analytics.

Tools Used:

Note Taking : Obsidian

Project Planning : Notion

Data Architecture Design : Draw.io

Table Naming Conventions

Bronze Rules

- All names must start with the source system name, and table names must match their original names without renaming.
- `<sourcesystem>_<entity>`
 - `<sourcesystem>` : Name of the source system (e.g., `crm` , `erp`).
 - `<entity>` : Exact table name from the source system.
 - Example: `crm_customer_info` → Customer information from the CRM system.

Silver Rules

- All names must start with the source system name, and table names must match their original names without renaming.
- `<sourcesystem>_<entity>`
- `<sourcesystem>` : Name of the source system (e.g., `crm` , `erp`).
- `<entity>` : Exact table name from the source system.
- Example: `crm_customer_info` → Customer information from the CRM system.

Gold Rules

- All names must use meaningful, business-aligned names for tables, starting with the category prefix.
- `<category>_<entity>`
- `<category>` : Describes the role of the table, such as `dim` (dimension) or `fact` (fact table).
- `<entity>` : Descriptive name of the table, aligned with the business domain (e.g., `customers` , `products` , `sales`).
- Examples:
 - `dim_customers` → Dimension table for customer data.
- `fact_sales` → Fact table containing sales transactions.

Glossary of Category Patterns

Pattern	Meaning	Example(s)
<code>dim_</code>	Dimension table	<code>dim_customer</code> , <code>dim_product</code>
<code>fact_</code>	Fact table	<code>fact_sales</code>
<code>report_</code>	Report table	<code>report_customers</code> , <code>report_sales_monthly</code>

Column Naming Conventions

Surrogate Keys

- All primary keys in dimension tables must use the suffix `_key` .
- `<table_name>_key`
- `<table_name>` : Refers to the name of the table or entity the key belongs to.
- `_key` : A suffix indicating that this column is a surrogate key.
- Example: `customer_key` → Surrogate key in the `dim_customers` table.

Technical Columns

- All technical columns must start with the prefix `dwh_` , followed by a descriptive name indicating the column's purpose.
- `dwh_<column_name>`

- dwh : Prefix exclusively for system-generated metadata.
- <column_name> : Descriptive name indicating the column's purpose.
- Example: dwh_load_date → System-generated column used to store the date when the record was loaded.

Stored Procedure

- All stored procedures used for loading data must follow the naming pattern:
- load_<layer> .
- <layer> : Represents the layer being loaded, such as bronze , silver , or gold .
- Example:
- load_bronze → Stored procedure for loading data into the Bronze layer.
- load_silver → Stored procedure for loading data into the Silver layer.

Implementation :

1. Creating Database (Data Warehouse) :

- Switch to the respective main database using **USE** statement . In this case we are switching to the master database .
- Now create a new Database using **CREATE DATABASE 'database_name'**. And switch to this newly created database using **USE**.
- Now we need to create the structure ie., the SCHEMAS using **CREATE SCHEMA schema_name** . Don't forget to use **GO (Separates batches when working with multiple SQL statements , similar to a separator)** after each schema creation statement

```
SQLQuery1.sql - JD\...use (JD\jerem (71))*  X
-- 1. Create Database 'Datawarehouse'

USE master; -- Switches to the master database

CREATE DATABASE DataWarehouse; -- Creates a new database

USE DataWarehouse;

-- Creating Schemas
CREATE SCHEMA bronze;
GO

CREATE SCHEMA silver;
GO

CREATE SCHEMA gold;
GO
```

- Remember , once you created an database and schema if you try to run the query again it will show an error . So write a **IF EXISTS ()** Statement to check whether the Database is already there , if it is present already **DROP** the table and create a new one else just create a new database.

```

--Drop and Recreate the 'Datawarehouse' Database
IF EXISTS (SELECT 1 FROM sys.databases WHERE name = 'DataWarehouse')
BEGIN
    ALTER DATABASE DataWarehouse SET SINGLE_USER WITH ROLLBACK IMMEDIATE;
    DROP DATABASE DataWarehouse;
END;
GO

--Create the 'Datawarehouse' Database
CREATE DATABASE DataWarehouse; -- Creates a new database
GO

USE DataWarehouse;
GO

```

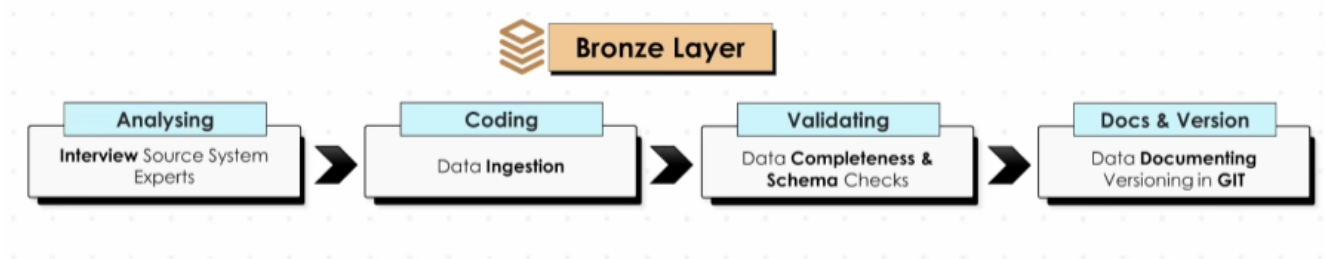
****NOTE:**

Always While writing a script make sure you give the script purpose description & WARNINGS in top of the script as a comment**

2. Building Bronze Layer :

-> Analyzing Source System

- Before building the bronze layer , we need to analyze the source system . Ask Several questions to the client / business to understand about the source system and the type of data
- Then we will be Coding , and the main purpose/outcome here is mostly **Data Ingestion**. That is deciding how to push/load the data from the source system to data warehouse.
- After loading the data , we need to do **Data Validation**, ie., quality checks . For example we need to compare and check whether the rows and column numbers are equal that of source system or is there any rows or columns that have been dropped or not
- Then we need to document what we have done in the process for better understanding.



How to understand the source systems ? (VERY IMPORTANT)

- Setup a meeting with the source system experts and ask them about the nature of the data in the source systems **(Below are the most common Questions we need to ask)**
 - **About Business Context & Ownership**
 1. Who owns the Data ?
 2. What Business Process it supports ? (does it support the customer transactions , supply chains etc.,)
 3. System and Data Documentation ?

4. Ask and Try to Understand the Data Model & Data Catalog (Descriptions about the table and columns)
- ****About Architecture & Technology Stack**
 1. How is the data stored in the source system ? (whether in SQL Server , Oracle , AWS , Azure etc etc)
 2. What are the integration capabilities ? (Can we connect through API , Kafka , File Extract , Direct DBEtc.,)
 - ****About Extract & Load**
 1. Can we do Incremental Loads or Full Loads ?
 2. What is the Data Scope & Historizations required ? (Whether we they want to analyze all the data or only the last 10 years of data etc)
 3. What is the expected size of the extracts ? (Whether the data is in Mb , Gb or in Tb)
 4. Are there any data volume limitations ?
 5. How to avoid impacting the source system performance ?
 6. What are the Authentication and Authorization , like how are we gonna access the data (through tokens , SSH Keys , VPN , IP Whitelisting etc)

-> Data Ingestion

Create DDL for Tables :

- After creating Schemas and understanding the Data we need to check how the data is in the source system (ie., Whether they have column names , are in right data type etc)
- Then create the table in the bronze layer as same as the table (ie., same exact header names and data types)
- NOTE : Kindly check and Maintain the naming conventions
- Create the Table and the necessary columns with their Datatype :

```
-- Creating Table for Source CRM - Customer Info
CREATE TABLE bronze.crm_cust_info (
    cst_id          INT,
    cst_key         NVARCHAR(50),
    cst_firstname   NVARCHAR(50),
    cst_lastname    NVARCHAR(50),
    cast_marital_status NVARCHAR(50),
    cst_gndr        NVARCHAR(50),
    cst_create_date DATE
)

-- Creating Table for Source CRM - Product Info
CREATE TABLE bronze.crm_prd_info (
    prd_id          INT,
    prd_key         NVARCHAR(50),
    prd_nm          NVARCHAR(50),
    prd_cost        INT,
    prd_line        NVARCHAR(50),
    prd_start_dt    DATETIME,
    prd_end_dt      DATETIME
)
```

- **NOTE :** Once the Table is created , if you run the query again it will throw an error saying the table already exists. Hence we need to put an **IF OBJECT ID ()** Statement to first check whether the table is exists or not

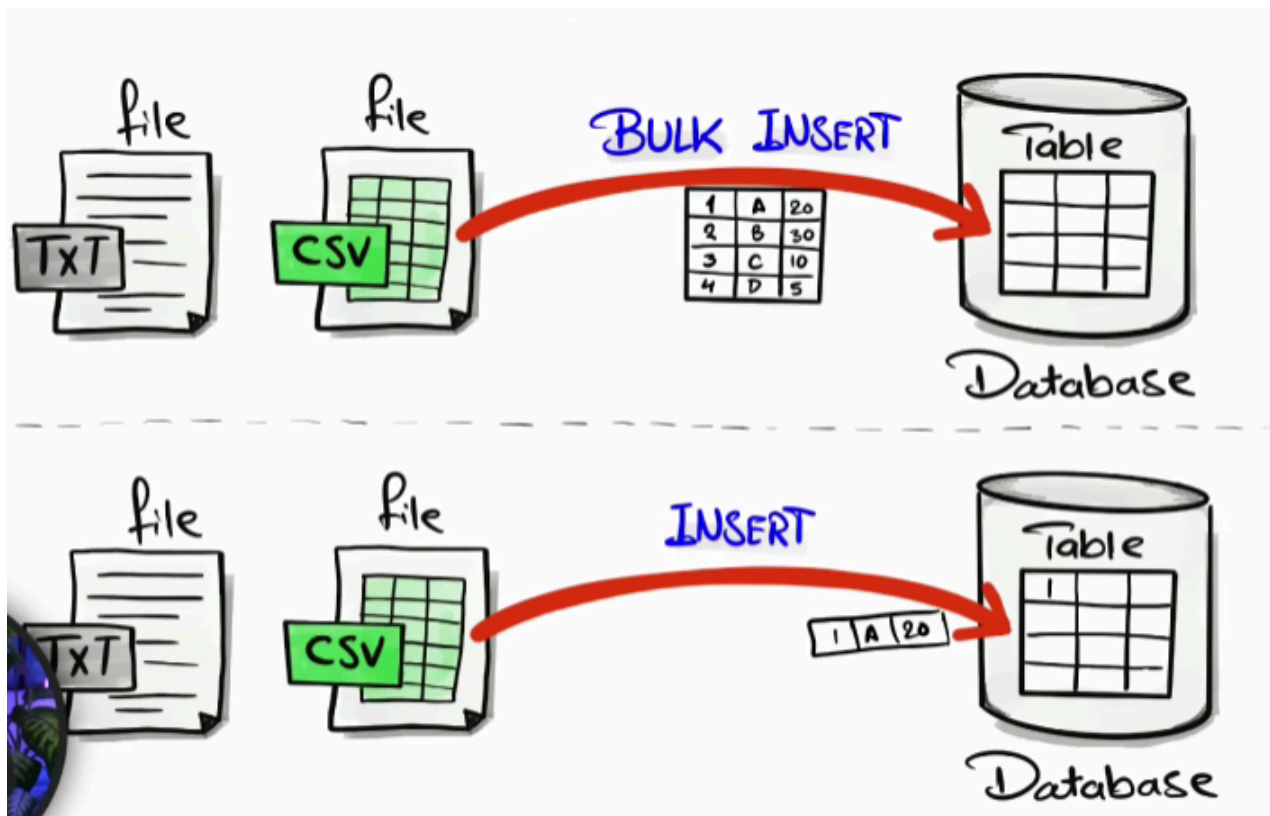
```
-- Checking whether the Object already exists or not
-- Syntax:
-- IF OBJECT_ID (<'object name/table name'> , <'object type'>) condition
-- argument
IF OBJECT_ID ('bronze.crm_cust_info' , 'U') IS NOT NULL
    DROP TABLE bronze.crm_cust_info;
-- Creating Table for Source CRM - Customer Info
CREATE TABLE bronze.crm_cust_info (
    cst_id            INT,
    cst_key           NVARCHAR(50),
    cst_firstname     NVARCHAR(50),
    cst_lastname      NVARCHAR(50),
    cast_marital_status NVARCHAR(50),
    cst_gndr          NVARCHAR(50),
    cst_create_date   DATE
)

IF OBJECT_ID ('bronze.crm_prd_info' , 'U') IS NOT NULL
    DROP TABLE bronze.crm_prd_info;
-- Creating Table for Source CRM - Product Info
CREATE TABLE bronze.crm_prd_info (
    prd_id            INT,
    prd_key           NVARCHAR(50),
    prd_nm            NVARCHAR(50),
    prd_cost          INT,
    prd_line          NVARCHAR(50),
    prd_start_dt      DATETIME,
    prd_end_dt        DATETIME
)
```

-> Develop SQL Load Scripts :

Loading the Data :

- Here we are gonna use **BULK INSERT** , bulk insert help to load the data very quickly from files like **.CSV** or **.TXT** directly to a database .
- The difference between **INSERT & BULK INSERT** is that the INSERT query will insert the data row by row , one by one but BULK INSERT will insert the entire data in one go thus making it faster than insert



- Insert the Data using **BULK INSERT** & Specify the location of the file present using **FROM** statement , Syntax :

```
SQLQuery3.sql - JD\...use (JD\jerem (53))  SQLQuery2.sql - JD\...use (JD\jerem (68))*  X  Creating Bronze lay...use (JD\jerem (51))
BULK INSERT bronze.crm_cust_info
FROM 'H:\Study Material\NOTES\Notes\Data Warehouse Project 1\sql-data-warehouse-project\datasets\source_crm\cust_info.csv'
```

- If there is a header/column name in the data , we need to extract the data from the 2nd row as the column name is not the data we need to start from . Important SQL keywords to remember her are WITH (

FIRSTROW = 2 --> 'row_number_from where the data should start from '

FIELDTERMINATOR = ',' --> 'Mention the delimiter/File separator to split the data based on it (, | # ; " - These are some commonly used delimiters)'

TABLOCK --> It locks the entire table while loading it , we use it to enhance the loading process

```
SQLQuery3.sql - JD\...use (JD\jerem (53))  SQLQuery2.sql - JD\...use (JD\jerem (68))*  X  Creating Bronze lay...use (JD\jerem (51))
BULK INSERT bronze.crm_cust_info
FROM 'H:\Study Material\NOTES\Notes\Data Warehouse Project 1\sql-data-warehouse-project\datasets\source_crm\cust_info.csv'
WITH (
    FIRSTROW = 2,
    FIELDTERMINATOR = ',',
    TABLOCK
)
100 %
Messages
(18493 rows affected)
Completion time: 2026-02-13T10:55:45.9548587+05:30
```

(VERY IMPORTANT)

- After this do a **==Quality Check Always==** , Check whether the data has been imported and check whether the data has not shifted from its respective columns and is in the correct column or not.

- NOTE : When you check the count of the data , there will always be 1 row less than the actual because in the csv file header will also be considered as a row but the sql doesn't

Adding Truncate Statement :

- and if we execute the Bulk insert one more time it will insert the data again on the existing data making it double the data . Hence we have to create argument to **TRUNCATE - Quickly Deletes all the row from the table , resetting the table to empty state** the existing data first and doing the BULK INSERT

```
SQLQuery2.sql - JD\...use (JD\jerem (68))* -p X SQLQuery3.sql - JD\...use (JD\jerem (53))
TRUNCATE TABLE bronze.crm_cust_info;
BULK INSERT bronze.crm_cust_info
FROM 'H:\Study Material\NOTES\Notes\Data Warehouse Project 1\sql-data-wa
WITH (
    FIRSTROW = 2,
    FIELDTERMINATOR = ',',
    TABLOCK
)
SELECT COUNT (*) FROM bronze.crm_cust_info
```

Creating Stored Procedure :

- Now , we might need to run this truncate and loading process every day to load the new data that is coming very data , **So to use a frequently used SQL Code we can store it in a stored procedure in a database**

```
Bronze - BULK INSE...use (JD\jerem (68))* -p X SQLQuery3.sql - JD\...use (JD\jerem (53))
CREATE OR ALTER PROCEDURE bronze.load_bronze AS
BEGIN
```

(Give the END at the bottom of the sql query)

- Now we can EXECUTE the stored procedure created

```
Bronze - BULK INSE...use (JD\jerem (68))* SQLQuery3.sql - JD\...use (JD\jerem (53))* -p X
EXECUTE bronze.load_bronze
```

100 %

Messages

```
(18493 rows affected)
(397 rows affected)
(60398 rows affected)
(18483 rows affected)
(18484 rows affected)
(37 rows affected)
```

Beautifying the Script:

- Once all the core SQL Queries are done , Always editing the output message for better readability using **PRINT 'text'**

```

Bronze - BULK INSE...use (JD\jerem (68))  X Bronze Stored Proce...use (JD\jerem (53))*
CREATE OR ALTER PROCEDURE bronze.load_bronze AS
BEGIN

    PRINT '=====';
    PRINT 'LOADING BRONZE LAYER';
    PRINT '=====';

    PRINT '*****';
    PRINT 'Loading CRM Tables';
    PRINT '*****';

    -- Bronze CRM Customer Info
    PRINT '>> Truncating the Table: bronze.crm_cust_info'
    TRUNCATE TABLE bronze.crm_cust_info;

    PRINT '>> Inserting Date Into: bronze.crm_cust_info'
    BULK INSERT bronze.crm_cust_info
    FROM 'H:\Study Material\NOTES\Notes\Data Warehouse Project 1\sql-data-war
    WITH (
        FIRSTROW = 2,
        FIELDTERMINATOR = ',',
        TABLOCK
    )

```

Error Handling using TRY....CATCH :

- Then , try to add **TRY ... CATCH** for Error Handling , data integrity and issue logging for easier debugging . SQL Will run the Code in between the **TRY Block** and if it fails it runs the **CATCH Block** to Handle the Error
- Inside CATCH , we need write what to do if the TRY Block faces any error . Either we can print a output statement saying there is an error or we can have a logging table with all the errors recorded there.

```

END TRY

BEGIN CATCH
    PRINT '*****';
    PRINT 'ERROR OCCURED DURING LOADING BRONZE LAYER';
    PRINT 'Error Message' + ERROR_MESSAGE;
    PRINT 'ERROR Message' + CAST (ERROR_NUMBER() AS NVARCHAR);
    PRINT 'ERROR Message' + CAST (ERROR_STATE() AS NVARCHAR);
    PRINT '*****';
END CATCH

END

```

- Main Code will be inside **BEGIN TRY END TRY**

Track ETL Duration:

- We need to track how long a step/process does take for executing it . This helps to identify bottlenecks, Optimize performance, monitor trends & detect issues
- First Declare the necessary variables using **DECLARE @variable_name Variable_Datatype**

```

Bronze - BULK INSE...use (JD\jerem (68))  X Bronze Stored Proce...use (JD\jerem (53))*
CREATE OR ALTER PROCEDURE bronze.load_bronze AS
BEGIN

    DECLARE @start_time DATETIME, @end_time DATETIME;
    BEGIN TRY

```

- Once Declared , use **SET** to start counting the time for executing a specific block and don't forget to print that with **DATEDIFF**

```

CREATE OR ALTER PROCEDURE bronze.load_bronze AS
BEGIN
    DECLARE @start_time DATETIME, @end_time DATETIME;
    BEGIN TRY

        PRINT '=====';
        PRINT 'LOADING BRONZE LAYER';
        PRINT '=====';

        PRINT '*****';
        PRINT 'Loading CRM Tables';
        PRINT '*****';

        SET @start_time = GETDATE();
        -- Bronze CRM Customer Info
        PRINT '>> Truncating the Table: bronze.crm_cust_info'
        TRUNCATE TABLE bronze.crm_cust_info;

        PRINT '>> Inserting Date Into: bronze.crm_cust_info'
        BULK INSERT bronze.crm_cust_info
        FROM 'H:\Study Material\notes\notes\Data Warehouse Project 1\sql-data-warehouse-project\datasets\source
        WITH (
            FIRSTROW = 2,
            FIELDTERMINATOR = ',',
            TABLOCK
        )
        SET @end_time = GETDATE();
        PRINT '>> LOAD DURATION : ' + CAST(DATEDIFF(second, @start_time, @end_time) AS NVARCHAR) + ' seconds'
        PRINT '>> -----'
    END TRY
    BEGIN CATCH
        PRINT 'Error: ' + ERROR_MESSAGE()
    END CATCH
END

```

- Since we are printing a string statement , but DATEDIFF will give output in numerical , we need to Type Cast it to a string . Hence , **CAST(DATEDIFF(type_of_timing, start_variable, end_varibale) AS NVARCHAR)**
- Now , we need to Duration timing for the whole batch , that is for the entire process.

- FINAL OUTPUT:

```
100 %
Messages

=====
-----LOADING BRONZE LAYER-----
=====
*****
Loading CRM Tables
*****
>> -----
>> Truncating the Table: bronze.crm_cust_info
>> Inserting Date Into: bronze.crm_cust_info

(18493 rows affected)
>> LOAD DURATION : 0 seconds
>> -----
>> Truncating the Table: bronze.crm_prd_info
>> Inserting Date Into: bronze.crm_prd_info

(397 rows affected)
>> LOAD DURATION : 0 seconds
>> -----
>> Truncating the Table: bronze.crm_sales_details
>> Inserting Date Into: bronze.crm_sales_details

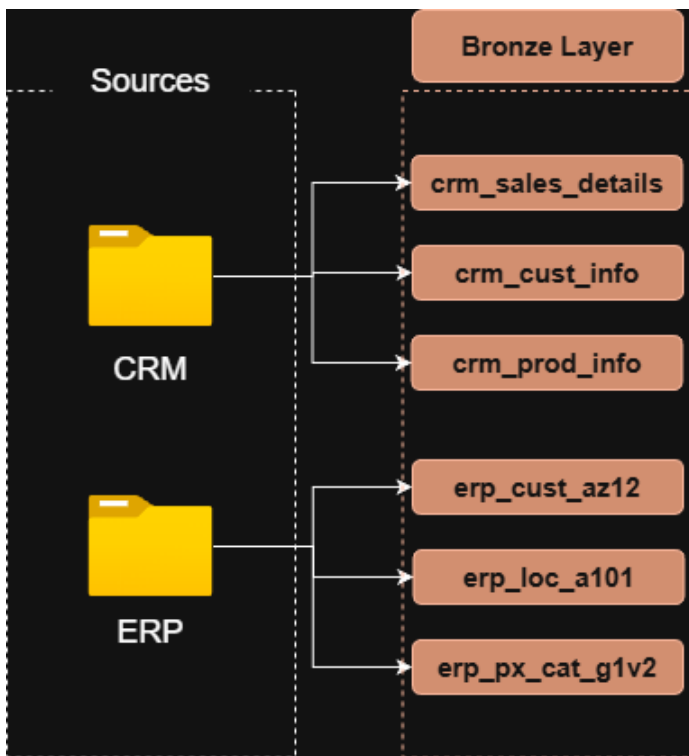
(60398 rows affected)
>> LOAD DURATION : 0 seconds
>> -----
*****
Loading ERP Tables
*****
>> -----
>> Truncating the Table: bronze.exp_cust_az12
>> Inserting Date Into: bronze.exp_cust_az12

(18483 rows affected)
>> LOAD DURATION : 0 seconds
>> -----
>> Truncating the Table: bronze.exp_loc_a101
>> Inserting Date Into: bronze.exp_loc_a101

(18484 rows affected)
>> LOAD DURATION : 0 seconds
>> -----
>> Truncating the Table: bronze.exp_px_cat_glv2
>> Inserting Date Into: bronze.exp_px_cat_glv2

(37 rows affected)
>> LOAD DURATION : 0 seconds
>> -----
>> DATA INJECTION INTO BRONZE LAYER COMPLETED
>> TOTAL LOAD DURATION OF BRONZE LAYER: 0 seconds
```

Data Flow After setting up bronze Layer



3. Building Silver Layer :

Now to build the Silver Layer and what is the Process in it ,

1. We have to Analyze the Data --> Explore and Understand the Data
 - In Bronze Layer we just built the layer and pushed the data inside , now we have take a moment and check and analyze the data for any issues or inconsistencies.
2. Start Coding --> Data Cleansing , We can do this by 3 steps :
 - Check the data quality issues of Bronze --> Check what the issues and inconsistencies
 - Write Data Transformations --> Fix the issues or inconsistencies one-by-one
 - Insert the Cleaned Data into Silver Layer
3. Validate the Data --> Perform Data Correctness Checks , This is like a cycle if we need to correct anything we will again go to 2nd step and again perform validation of data
4. Document and Commit in Git --> Data Flow & Data Integration Diagram

-> Analyzing Data in Bronze Layer :

- Just as you go don't try to check the entire table of data , Limit you results and check for top 1000 or 5000 values

```

SELECT TOP (1000) [cst_id]
, [cst_key]
, [cst_firstname]
, [cst_lastname]
, [cast_marital_status]
, [cst_gndr]
, [cst_create_date]
FROM [DataWarehouse].[bronze].[crm_cust_info]

```

100 %

Results Messages

	cst_id	cst_key	cst_firstname	cst_lastname	cast_marital_status	cst_gndr	cst_create_date
1	11000	AW00011000	Jon	Yang	M	M	2025-10-06
2	11001	AW00011001	Eugene	Huang	S	M	2025-10-06
3	11002	AW00011002	Ruben	Torres	M	M	2025-10-06
4	11003	AW00011003	Christy	Zhu	S	F	2025-10-06
5	11004	AW00011004	Elizabeth	Johnson	S	F	2025-10-06
6	11005	AW00011005	Julio	Ruiz	S	M	2025-10-06
7	11006	AW00011006	Janet	Alvarez	S	F	2025-10-06
8	11007	AW00011007	Marco	Mehta	M	M	2025-10-06
9	11008	AW00011008	Rob	Verhoff	S	F	2025-10-06
10	11009	AW00011009	Shannon	Carlson	S	M	2025-10-06
11	11010	AW00011010	Jacquelyn	Suarez	S	F	2025-10-06
12	11011	AW00011011	Curtis	Lu	M	M	2025-10-06
13	11012	AW00011012	Lauren	Walker	M	F	2025-10-06

- Keep a Diagram designer application open , to build the Data Model or a Integration Model and how to integrate it , (Draw.io)
- Create table diagram and mention the table name that we are analyzing as of now,
 - Mention through which we will be connecting the tables or aggregating new tables from it
 - Example : Primary Key of the table (cst_id) , Secondary Key if it has (cst_key)
 - Likewise Check for each table and try to connect and model a architecture
 - If you see in Sales Details Table we have (sls_prd_key , sls_cust_id) which are the Primary Keys in previous CRM tables , Hence we can connect this table to those tables. These type of Table is called **Event and Transactional Tables**

```

SELECT TOP 1000 * FROM bronze.crm_cust_info
SELECT TOP 1000 * FROM bronze.crm_prd_info
SELECT TOP 1000 * FROM bronze.crm_sales_details

```

Event and Transactional Table for Sales !

Can be used to connect with other tables !

	sls_ord_num	sls_prd_key	sls_cust_id	sls_order_dt	sls_ship_dt	sls_due_dt	sls_sales	sls_quantity
1	SO43697	BK-R93R-62	21768	20101229	20110105	20110110	3578	1
2	SO43698	BK-M82S-44	28389	20101229	20110105	20110110	3400	1
3	SO43699	BK-M82S-44	25863	20101229	20110105	20110110	3400	1
4	SO43700	BK-R50B-62	14501	20101229	20110105	20110110	699	1
5	SO43701	BK-M82S-44	11003	20101229	20110105	20110110	3400	1
6	SO43702	BK-R93R-44	27645	20101230	20110106	20110111	3578	1
7	SO43703	BK-R93R-62	16624	20101230	20110106	20110111	3578	1
8	SO43704	BK-M82B-48	11005	20101230	20110106	20110111	3375	1
9	SO43705	BK-M82S-38	11011	20101230	20110106	20110111	3400	1
10	SO43706	BK-R93R-48	27621	20101231	20110107	20110112	3578	1

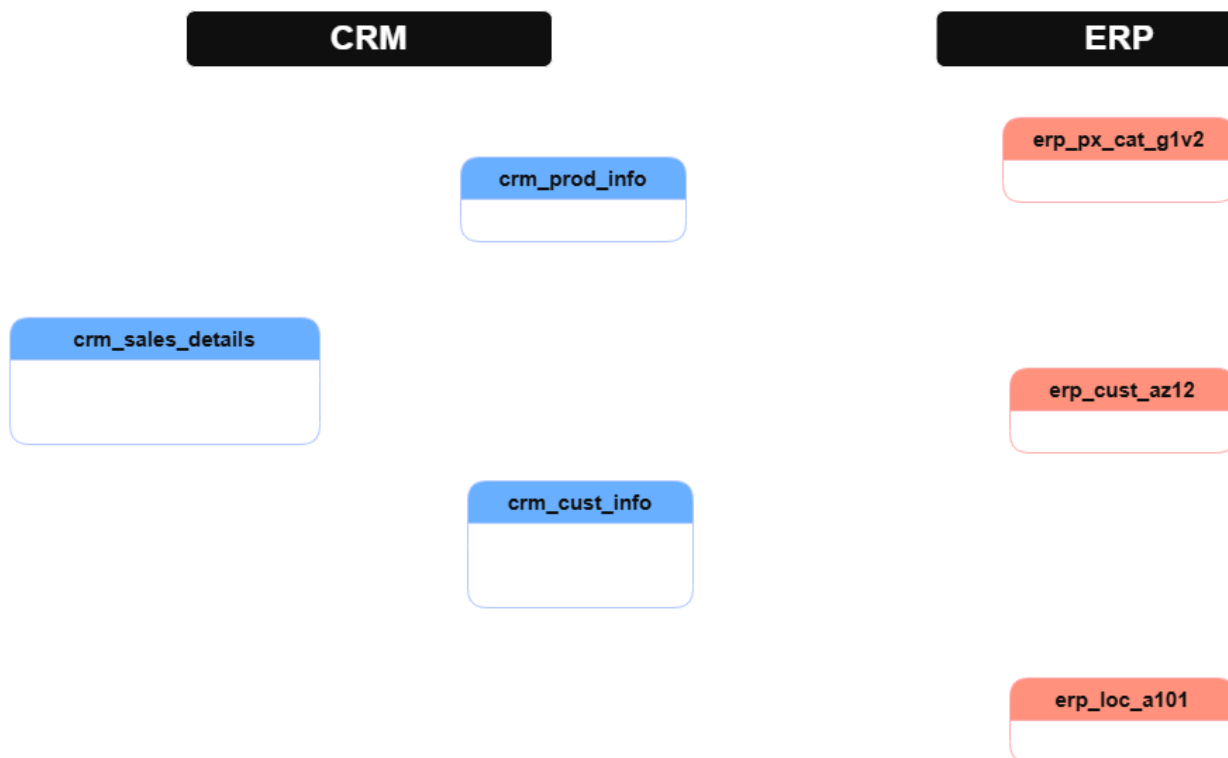
- Sometimes we might have to derive half of the key to match the primary key of another table to connect those 2 related tables.

```
SELECT TOP 1000 * FROM bronze.erp_px_cat_g1v2;
SELECT TOP 1000 * FROM bronze.crm_prd_info;
```

id	cat	subcat	maintenance
AC_BR	Accessories	Bike Racks	Yes
AC_BS	Accessories	Bike Stands	No
AC_BC	Accessories	Bottles and Cages	No
AC_CL	Accessories	Cleaners	Yes
AC_FE	Accessories	Fenders	No
AC_HE	Accessories	Helmets	Yes
AC_HP	Accessories	Hydration Packs	No
AC_LI	Accessories	Lights	Yes

prd_id	prd_key	prd_nm	prd_cost
210	CO-RF-FR-R92B-58	HL Road Frame - Black- 58	NULL
211	CO-RF-FR-R92R-58	HL Road Frame - Red- 58	NULL
212	AC HE HL 11500 B	Sport 100 Helmet Red	12

- This is the Final Data Model on how we will be connecting the Data and all the tables in the source systems.



-> Create DDL for Silver Layer tables :

- To create tables for Silver layer we can take the DDL Script of Bronze layer and just replace bronze. --> silver. in the script. Use (Ctrl+H) to use the tool find and replace.

- But before executing check whether we need any **Metadata Columns** ,
- Metadata Columns are additional / Extra columns or fields that are added by Data Engineers that do not originate from the source data. For example :
 - create_date --> The Record's load timestamp
 - update_date --> the records last update timestamp
 - source_system --> The Origin System of the Record
 - file_location --> The file Source of the Record
- (This might help you when a error or any data corruption happens)

```
-- Checking whether the Object already exists or not
-- Syntax:
-- IF OBJECT_ID (<'object name/table name'> , <'object type'>) condition
-- argument
IF OBJECT_ID ('silver.crm_cust_info' , 'U') IS NOT NULL
    DROP TABLE silver.crm_cust_info;
-- Creating Table for Source CRM - Customer Info
CREATE TABLE silver.crm_cust_info (
    cst_id            INT,
    cst_key            NVARCHAR(50),
    cst_firstname      NVARCHAR(50),
    cst_lastname       NVARCHAR(50),
    cast_marital_status NVARCHAR(50),
    cst_gndr           NVARCHAR(50),
    cst_create_date    DATE,
    dwh_create_date    DATETIME2 DEFAULT GETDATE()
)

IF OBJECT_ID ('silver.crm_prd_info' , 'U') IS NOT NULL
    DROP TABLE silver.crm_prd_info;
-- Creating Table for Source CRM - Product Info
CREATE TABLE silver.crm_prd_info (
    prd_id            INT,
    prd_key            NVARCHAR(50),
    prd_nm             NVARCHAR(50),
    prd_cost           INT,
    prd_line           NVARCHAR(50),
    prd_start_dt       DATETIME,
    prd_end_dt         DATETIME,
    dwh_create_date    DATETIME2 DEFAULT GETDATE()
)
```

-> Cleaning & Loading the Data :

- Before Pushing to Silver Layer , Check the issues using the bronze layer data one by one

1. crm_cust_info

1. Check for Nulls or Duplicates in Primary Key :

- Check the Primary Key for any Duplicates or any Null values are present are not Using **HAVING COUNT > 1**

SQLQuery11.sql - JD...use (JD\jerem (72))* Creating Silver laye...ouse (JD\jerem (71))

```

SELECT
  cst_id,
  COUNT(*)
FROM bronze.crm_cust_info

-- Checks and Groups only the values that are having more than 1 Count
GROUP BY cst_id
HAVING COUNT(*) > 1 OR cst_id IS NULL

```

100 %

Results Messages

	cst_id	(No column name)
1	29449	2
2	29473	2
3	29433	2
4	NULL	3
5	29483	2
6	29466	3

- Take a specific value which has duplicate values and check what are the duplicates , here the table has history values so the same data for past 3 years are present . Hence the duplicate values , If you have history values ... Mostly we will be picking the latest value.

```

SELECT
  *
FROM bronze.crm_cust_info
WHERE cst_id = 29466

```

100 %

Results Messages

	cst_id	cst_key	cst_firstname	cst_lastname	cast_marital_status	cst_gndr	cst_create_date
1	29466	AW00029466	NULL	NULL	NULL	NULL	2026-01-25
2	29466	AW00029466	Lance	Jimenez	M	NULL	2026-01-26
3	29466	AW00029466	Lance	Jimenez	M	M	2026-01-27

- Here we have only 3 values , but incase if we have many values like 100 values it would be better if we arrange the data in descending order (From highest to Lowest) for that we can Use **WINDOW Function**
- Syntax = ROW_NUMBER() OVER (PARTITION BY column_ID ORDER BY column_name_by_which_ORDERBY_should_happen) as new_Col_name
- ROW_NUMBER() OVER (PARTITION BY cst_id ORDER BY cst_create_date DESC) as flag_last

```
-- Script to remove the duplicates in Primary Key
```

```
SELECT
*,
ROW_NUMBER() OVER (PARTITION BY cst_id ORDER BY cst_create_date DESC) as flag_last
FROM bronze.crm_cust_info
WHERE cst_id = 29466
```

	cst_id	cst_key	cst_firstname	cst_lastname	cast_marital_status	cst_gndr	cst_create_date	flag_last
1	29466	AW00029466	Lance	Jimenez	M	M	2026-01-27	1
2	29466	AW00029466	Lance	Jimenez	M	NULL	2026-01-26	2
3	29466	AW00029466	NULL	NULL	NULL	NULL	2026-01-25	3

- Now we have identified the inconsistency and checked for 1 data we can now remove the filter for a specific data and apply the query for entire table

```
-- Script to remove the duplicates in Primary Key
```

```
SELECT
*,
ROW_NUMBER() OVER (PARTITION BY cst_id ORDER BY cst_create_date DESC) as flag_last
FROM bronze.crm_cust_info
```

	cst_id	cst_key	cst_firstname	cst_lastname	cast_marital_status	cst_gndr	cst_create_date	flag_last
1	NULL	PO25	NULL	NULL	NULL	NULL	NULL	1
2	NULL	SF566	NULL	NULL	NULL	NULL	NULL	2
3	NULL	13451235	NULL	NULL	NULL	NULL	NULL	3
4	11000	AW00011000	Jon	Yang	M	M	2025-10-06	1
5	11001	AW00011001	Eugene	Huang	S	M	2025-10-06	1
6	11002	AW00011002	Ruben	Torres	M	M	2025-10-06	1
7	11003	AW00011003	Christy	Zhu	S	F	2025-10-06	1
8	11004	AW00011004	Elizabeth	Johnson	S	F	2025-10-06	1
9	11005	AW00011005	Julio	Ruiz	S	M	2025-10-06	1
10	11006	AW00011006	Janet	Alvarez	S	F	2025-10-06	1
11	11007	AW00011007	Marco	Mehta	M	M	2025-10-06	1
12	11008	AW00011008	Rob	Verhoff	S	F	2025-10-06	1
13	11009	AW00011009	Shannon	Carlson	S	M	2025-10-06	1
14	11010	AW00011010	Jacquelyn	Suarez	S	F	2025-10-06	1
15	11011	AW00011011	Curtis	Lu	M	M	2025-10-06	1
16	11012	AW00011012	Lauren	Walker	M	F	2025-10-06	1
17	11013	AW00011013	Ian	Jenkins	M	M	2025-10-06	1
18	11014	AW00011014	Sydney	Bennett	S	F	2025-10-06	1
19	11015	AW00011015	Chloe	Young	S	F	2025-10-06	1
20	11016	AW00011016	Wyatt	Hill	M	M	2025-10-07	1
21	11017	AW00011017	Shannon	Wang	S	F	2025-10-07	1
22	11018	AW00011018	Clarence	Rai	S	M	2025-10-07	1

- Put the code inside another SELECT func WHERE flag_last != 1 to filter out the values having more than 1

```
-- Script to remove the duplicates in Primary Key
```

```
SELECT
*
FROM (
  SELECT
  *,
  ROW_NUMBER() OVER (PARTITION BY cst_id ORDER BY cst_create_date DESC) as flag_last
  FROM bronze.crm_cust_info
)
t WHERE flag_last !=1
```

100 %

Results Messages

	cst_id	cst_key	cst_firstname	cst_lastname	cast_marital_status	cst_gndr	cst_create_date	flag_last
1	NULL	SF566	NULL	NULL	NULL	NULL	NULL	2
2	NULL	13451235	NULL	NULL	NULL	NULL	NULL	3
3	29433	AW00029433	NULL	NULL	M	M	2026-01-25	2
4	29449	AW00029449	NULL	Chen	S	NULL	2026-01-25	2
5	29466	AW00029466	Lance	Jimenez	M	NULL	2026-01-26	2
6	29466	AW00029466	NULL	NULL	NULL	NULL	2026-01-25	3
7	29473	AW00029473	Camren	NULL	NULL	NULL	2026-01-25	2
8	29483	AW00029483	NULL	Navarro	NULL	NULL	2026-01-25	2

- Now we need only the values with flag_last value = 1 Data and need to leave of the data of 2 , 3. Just change the !=1 to =1


```
-- Script to remove the duplicates in Primary Key
```

```
SELECT
*
FROM (
  SELECT
  *,
  ROW_NUMBER() OVER (PARTITION BY cst_id ORDER BY cst_create_date DESC) as flag_last
  FROM bronze.crm_cust_info
)
t WHERE flag_last = 1
```

	cst_id	cst_key	cst_firstname	cst_lastname	cst_marital_status	cst_gndr	cst_create_date	flag_last
1	NULL	PO25	NULL	NULL	NULL	NULL	NULL	1
2	11000	AW00011000	Jon	Yang	M	M	2025-10-06	1
3	11001	AW00011001	Eugene	Huang	S	M	2025-10-06	1
4	11002	AW00011002	Ruben	Torres	M	M	2025-10-06	1
5	11003	AW00011003	Christy	Zhu	S	F	2025-10-06	1
6	11004	AW00011004	Elizabeth	Johnson	S	F	2025-10-06	1
7	11005	AW00011005	Julio	Ruiz	S	M	2025-10-06	1
8	11006	AW00011006	Janet	Alvarez	S	F	2025-10-06	1
9	11007	AW00011007	Marco	Mehta	M	M	2025-10-06	1
10	11008	AW00011008	Rob	Verhoff	S	F	2025-10-06	1
11	11009	AW00011009	Shannon	Carlson	S	M	2025-10-06	1
12	11010	AW00011010	Jacquelyn	Suarez	S	F	2025-10-06	1
13	11011	AW00011011	Curtis	Lu	M	M	2025-10-06	1
14	11012	AW00011012	Lauren	Walker	M	F	2025-10-06	1
15	11013	AW00011013	Ian	Jenkins	M	M	2025-10-06	1
16	11014	AW00011014	Sydney	Bennett	S	F	2025-10-06	1
17	11015	AW00011015	Chloe	Young	S	F	2025-10-06	1
18	11016	AW00011016	Wyatt	Hill	M	M	2025-10-07	1

2. Check for Unwanted Spaces in String values :

- Removes unnecessary spaces to ensure data consistency, and uniformity across all records

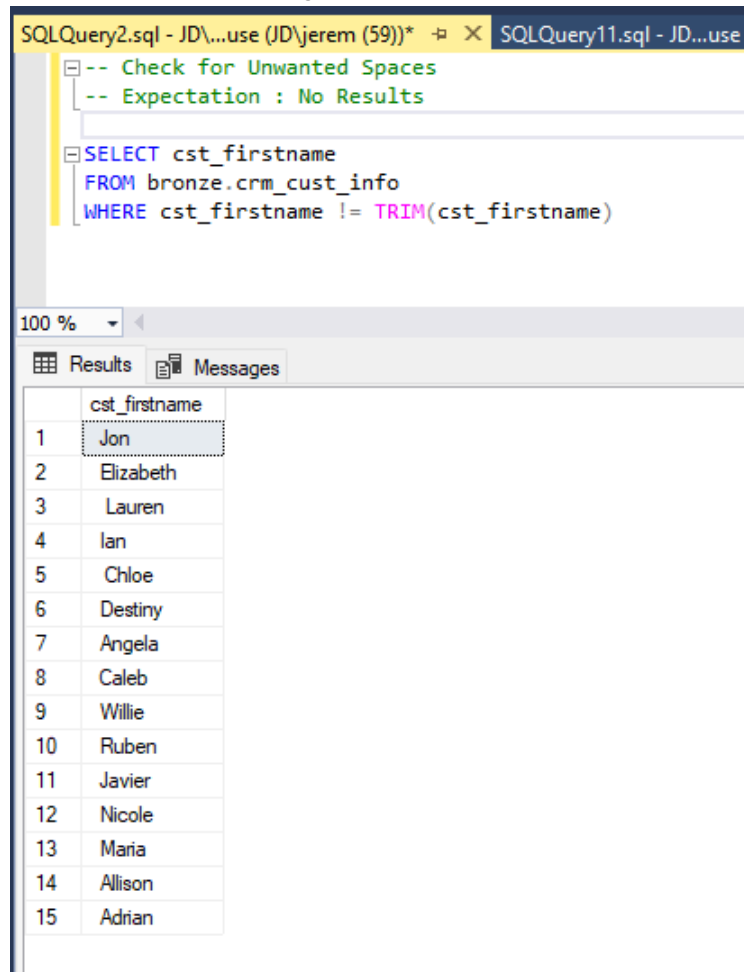
Quality Check

Check for unwanted spaces in string values

	cst_id	cst_key	cst_firstname	cst_lastname	cst_marital_status	cst_gndr	cst_create_date
1	11000	AW00011000	Jon	Yang	M	M	2025-10-06
2	11001	AW00011001	Eugene	Huang	S	M	2025-10-06
3	11002	AW00011002	Ruben	Torres	M	M	2025-10-06
4	11003	AW00011003	Christy	Zhu	S	F	2025-10-06
5	11004	AW00011004	Elizabeth	Johnson	S	F	2025-10-06
6	11005	AW00011005	Julio	Ruiz	S	M	2025-10-06
7	11006	AW00011006	Janet	Alvarez	S	F	2025-10-06
8	11007	AW00011007	Marco	Mehta	M	M	2025-10-06

- Sometimes we might have unwanted spaces in front and back of string values , we need to check those as well, But just by seeing in plain eyes we might not know whether there is a space after the string end

- Here , can use the **TRIM() Function** to remove the leading and trailing spaces from a string , (If the original value is not equal to the same value after trimming it means there are spaces)
- Likewise do for all string Columns



The screenshot shows a SQL Server Enterprise Manager interface. At the top, there are two tabs: 'SQLQuery2.sql - JD\...use (JD\jerem (59))*' and 'SQLQuery11.sql - JD...use'. The active query window contains the following SQL code:

```
-- Check for Unwanted Spaces
-- Expectation : No Results

SELECT cst_firstname
FROM bronze.crm_cust_info
WHERE cst_firstname != TRIM(cst_firstname)
```

Below the query window, there is a 'Results' tab and a 'Messages' tab. The 'Results' tab is active, showing a table with 15 rows and one column named 'cst_firstname'. The data in the table is as follows:

	cst_firstname
1	Jon
2	Elizabeth
3	Lauren
4	Ian
5	Chloe
6	Destiny
7	Angela
8	Caleb
9	Willie
10	Ruben
11	Javier
12	Nicole
13	Maria
14	Allison
15	Adrian

- We have checked and we got some data where there is extra spaces , Now we need to write a new query to rectify this and check again this query and we should get an output as No Results

Remove Unwanted S...se (JD\jerem (63))

Check for Data Stan...use (JD\jerem (54))

-- Remove the extra unwanted spaces

SELECT

cst_id,

cst_key,

TRIM(cst_firstname) AS cst_firstname,

TRIM(cst_lastname) AS cst_lastname,

cst_marital_status,

cst_gndr,

cst_create_date

FROM (

SELECT

*

ROW_NUMBER() OVER (PARTITION BY cst_id ORDER BY cst_create_date DESC) as flag_last

FROM bronze.crm_cust_info

WHERE cst_id IS NOT NULL

)

t WHERE flag_last = 1

100 %

Results

Messages

	cst_id	cst_key	cst_firstname	cst_lastname	cst_marital_status	cst_gndr	cst_create_date
1	11000	AW00011000	Jon	Yang	M	M	2025-10-06
2	11001	AW00011001	Eugene	Huang	S	M	2025-10-06
3	11002	AW00011002	Ruben	Torres	M	M	2025-10-06
4	11003	AW00011003	Christy	Zhu	S	F	2025-10-06
5	11004	AW00011004	Elizabeth	Johnson	S	F	2025-10-06
6	11005	AW00011005	Julio	Ruiz	S	M	2025-10-06
7	11006	AW00011006	Janet	Alvarez	S	F	2025-10-06
8	11007	AW00011007	Marco	Mehta	M	M	2025-10-06
9	11008	AW00011008	Rob	Verhoff	S	F	2025-10-06
10	11009	AW00011009	Shannon	Carlson	S	M	2025-10-06
11	11010	AW00011010	Jacquelyn	Suarez	S	F	2025-10-06
12	11011	AW00011011	Curtis	Lu	M	M	2025-10-06
13	11012	AW00011012	Lauren	Walker	M	F	2025-10-06
14	11013	AW00011013	Ian	Jenkins	M	M	2025-10-06
15	11014	AW00011014	Sydney	Bennett	S	F	2025-10-06
16	11015	AW00011015	Chloe	Young	S	F	2025-10-06
17	11016	AW00011016	Wyatt	Hill	M	M	2025-10-07
18	11017	AW00011017	Shannon	Wang	S	F	2025-10-07
19	11018	AW00011018	Clarence	Rai	S	M	2025-10-07

3. Need to check Data Cardinality / Standardization :

- It is nothing but changing the data into more understanding words / data . For Example : Here 2 columns have M one in cst_marital_status & cst_gndr ...but M can be Married or Male so we need to specifically change into Male in cst_gndr & Married in cst_marital_status.
- Likewise , it is better to use full words that to use abbreviations

Quality Check

Check the consistency of values in low cardinality columns

	cst_id	cst_key	cst_firstname	cst_lastname	cst_marital_status	cst_gndr	cst_create_date
92	11091	AW00011091	Dalton	Perez	M	M	2025-10-07
93	11092	AW00011092	Cheryl	Diaz	M	F	2025-10-07
94	11093	AW00011093	Aimee	He	M	F	2025-10-07
95	11094	AW00011094	Cedric	Ma	S	M	2025-10-07
96	11095	AW00011095	Chad	Kumar	S	M	2025-10-07
97	11096	AW00011096	Andr@s	Anand	M	M	2025-10-07
98	11097	AW00011097	Edwin	Nara	M	M	2025-10-07
99	11098	AW00011098	Mallory	Rubio	S	F	2025-10-07
100	11099	AW00011099	Adam	Ross	M	M	2025-10-07

	cst_gndr
1	NULL
2	F
3	M

In our data warehouse, we aim to store clear and meaningful values rather than using abbreviated terms

- Use CASE WHEN col_name = 'abbreviation' THEN 'Full word' END col_name , To change the abbreviated letters into meaningful words.

```
-- Data Standardization and removeing abbreviations
```

```
SELECT
  cst_id,
  cst_key,
  TRIM(cst_firstname) AS cst_firstname,
  TRIM(cst_lastname) AS cst_lastname,

  CASE WHEN cst_marital_status = 'S' THEN 'Single'
        WHEN cst_marital_status = 'M' THEN 'Married'
        ELSE 'N/A'
  END cst_marital_status,

  CASE WHEN cst_gndr = 'F' THEN 'Female'
        WHEN cst_gndr = 'M' THEN 'Male'
        ELSE 'N/A'
  END cst_gndr,

  cst_create_date
FROM (
  SELECT
    *,
    ROW_NUMBER() OVER (PARTITION BY cst_id ORDER BY cst_create_date DESC) as flag_last
  FROM bronze.crm_cust_info
  WHERE cst_id IS NOT NULL
)
t WHERE flag_last = 1
```

0 %

Results Messages

	cst_id	cst_key	cst_firstname	cst_lastname	cst_marital_status	cst_gndr	cst_create_date
1	11000	AW00011000	Jon	Yang	Married	Male	2025-10-06
2	11001	AW00011001	Eugene	Huang	Single	Male	2025-10-06
3	11002	AW00011002	Ruben	Tomes	Married	Male	2025-10-06
4	11003	AW00011003	Christy	Zhu	Single	Female	2025-10-06
5	11004	AW00011004	Elizabeth	Johnson	Single	Female	2025-10-06
6	11005	AW00011005	Julio	Ruiz	Single	Male	2025-10-06
7	11006	AW00011006	Janet	Alvarez	Single	Female	2025-10-06
8	11007	AW00011007	Marco	Mehta	Married	Male	2025-10-06
9	11008	AW00011008	Rob	Verhoff	Single	Female	2025-10-06
10	11009	AW00011009	Shannon	Carlson	Single	Male	2025-10-06
11	11010	AW00011010	Jacquelyn	Suarez	Single	Female	2025-10-06
12	11011	AW00011011	Curtis	Lu	Married	Male	2025-10-06

- Now sometimes the abbreviated Letters might be a mix of Upper case & Lower case letters , In that case we need to type convert all the Letters into Upper case and then convert to meaningful word for better understanding

```
CASE WHEN UPPER(cst_marital_status) = 'S' THEN 'Single'
      WHEN UPPER(cst_marital_status) = 'M' THEN 'Married'
      ELSE 'N/A'
END cst_marital_status,

CASE WHEN UPPER(cst_gndr) = 'F' THEN 'Female'
      WHEN UPPER(cst_gndr) = 'M' THEN 'Male'
      ELSE 'N/A'
END cst_gndr,
```

- Just to make sure there are no extra spaces in the Data we can add TRIM() as well

```

CASE WHEN UPPER(TRIM(cst_marital_status)) = 'S' THEN 'Single'
      WHEN UPPER(TRIM(cst_marital_status)) = 'M' THEN 'Married'
      ELSE 'N/A'
END cst_marital_status,

CASE WHEN UPPER(TRIM(cst_gndr)) = 'F' THEN 'Female'
      WHEN UPPER(TRIM(cst_gndr)) = 'M' THEN 'Male'
      ELSE 'N/A'
END cst_gndr,

```

4. Inserting into silver Layer After Basic Cleaning :

- Now insert the Data into Silver layer as far as how much cleaning we have done

```

-- Inserting into silver Layer After Basic Cleaning

INSERT INTO silver.crm_cust_info (
    cst_id,
    cst_key,
    cst_firstname,
    cst_lastname,
    cst_marital_status,
    cst_gndr,
    cst_create_date
)

SELECT
    cst_id,
    cst_key,
    TRIM(cst_firstname) AS cst_firstname,
    TRIM(cst_lastname) AS cst_lastname,

    CASE WHEN UPPER(TRIM(cst_marital_status)) = 'S' THEN 'Single'
          WHEN UPPER(TRIM(cst_marital_status)) = 'M' THEN 'Married'
          ELSE 'N/A'
    END cst_marital_status,

    CASE WHEN UPPER(TRIM(cst_gndr)) = 'F' THEN 'Female'
          WHEN UPPER(TRIM(cst_gndr)) = 'M' THEN 'Male'
          ELSE 'N/A'
    END cst_gndr,
    cst_create_date

```

100 %

Messages

(18484 rows affected)

Completion time: 2026-03-11T14:42:13.4879858+05:30

- Perform Quality checks in silver layer

Check for Extra Unw...use (JD\jerem (70))
Check for Duplicate...use (JD\jerem (84))
SQLQuery

```

--QUALITY CHECKS of SILVER LAYER

-- 1. Check for Null / Duplicates in Primary Key

SELECT
  cst_id,
  COUNT(*)
FROM silver.crm_cust_info

-- Checks and Groups only the values that are having more than 1 Count
GROUP BY cst_id
HAVING COUNT(*) > 1 OR cst_id IS NULL

-----

-- 2. Check for Unwanted Spaces

SELECT cst_firstname
FROM silver.crm_cust_info
WHERE cst_firstname != TRIM(cst_firstname)

-----

-- 3. Data Standardization & Consistency

SELECT DISTINCT cst_marital_status
FROM silver.crm_cust_info

```

100 %

Results
Messages

cst_id	(No column name)
--------	------------------

cst_firstname

cst_marital_status
1 Single
2 Married

- Likewise Check for all the csv data files.

2. crm_prd_info :

1. Check for Nulls or Duplicates in Primary Key :

```
-- Checking Duplicates in Primary Key

SELECT
    prd_id,
    COUNT(*)
FROM bronze.crm_prd_info
GROUP BY prd_id
HAVING COUNT(*) > 1 OR prd_id IS NULL
```

100 %

Results Messages

prd_id	(No column name)
--------	------------------

2. Splitting and deriving 2 New columns of Data :

- In this data the Col --> prd_key , contains two informations merged together that is Category Serial + Product Serial Codes --> CO-RF(Category)-FR-R92B-58(Product).
- Hence we need to split this to perform good data analysis , Here we Can use the **SUBSTRING Function** --> Extracts a Specific part of a string value , (It needs 3 arguments)
- Syntax : SUBSTRING (column_name_of_initial_Data, Starting_length, Ending_length) AS new_col name
- SUBSTRING (prd_key, 1, 5) AS Cate_id

```
SELECT
    prd_key,
    SUBSTRING(prd_key, 1, 5) AS cat_id
FROM bronze.crm_prd_info
```

0 %

Results Messages

	prd_key	cat_id
1	CO-RF-FR-R92B-58	CO-RF
2	CO-RF-FR-R92R-58	CO-RF
3	AC-HE-HL-U509-R	AC-HE
4	AC-HE-HL-U509-R	AC-HE
5	AC-HE-HL-U509-R	AC-HE
6	AC-HE-HL-U509	AC-HE
7	AC-HE-HL-U509	AC-HE
8	AC-HE-HL-U509	AC-HE


```

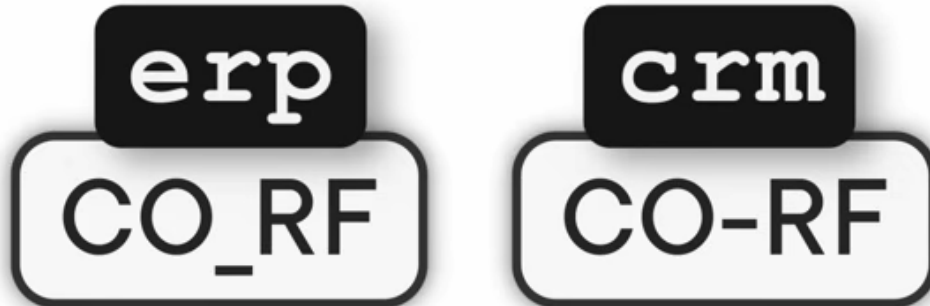
SELECT
    prd_key,
    SUBSTRING(prd_key, 1, 5) AS cat_key,
    SUBSTRING(prd_key, 7, 16) AS prod_key
FROM bronze.crm_prd_info

```

1 %		
Results Messages		
prd_key	cat_key	prod_key
CO-RF-FR-R92B-58	CO-RF	FR-R92B-58
CO-RF-FR-R92R-58	CO-RF	FR-R92R-58
AC-HE-HL-U509-R	AC-HE	HL-U509-R
AC-HE-HL-U509-R	AC-HE	HL-U509-R
AC-HE-HL-U509-R	AC-HE	HL-U509-R
AC-HE-HL-U509	AC-HE	HL-U509
AC-HE-HL-U509	AC-HE	HL-U509
AC-HE-HL-U509	AC-HE	HL-U509
CL-SO-SO-B909-M	CL-SO	SO-B909-M
CL-SO-SO-B909-L	CL-SO	SO-B909-L

3. Splitting and deriving 2 New columns of Data :

- If you notice we have a Category Id in one of the ERM tables , But 1 difference is there here the Category id = CO-RF , in ERM = CO_RF .
- So now we have to replace '-' with '_'



- For this we can use the Function **REPLACE()** ,
- Syntax : **REPLACE (Col_name, 'What to Replace', 'to Replace With')**
 - **REPLACE (catkey, '-', '_')**

```

SELECT
    prd_key,
    REPLACE(prd_key, '-', '_')
FROM bronze.crm_prd_info

```

100 %

	prd_key	(No column name)
1	CO-RF-FR-R92B-58	CO_RF_FR_R92B_58
2	CO-RF-FR-R92R-58	CO_RF_FR_R92R_58
3	AC-HE-HL-U509-R	AC_HE_HL_U509_R
4	AC-HE-HL-U509-R	AC_HE_HL_U509_R
5	AC-HE-HL-U509-R	AC_HE_HL_U509_R
6	AC-HE-HL-U509	AC_HE_HL_U509
7	AC-HE-HL-U509	AC_HE_HL_U509
8	AC-HE-HL-U509	AC_HE_HL_U509

```

SELECT prd_id,
    prd_key,
    REPLACE(SUBSTRING(prd_key, 1, 5), '-', '_') AS cat_key,
    SUBSTRING(prd_key, 7, 16) AS prod_key,
    prd_nm,
    prd_cost,
    prd_line,
    prd_start_dt,
    prd_end_dt
FROM bronze.crm_prd_info

```

100 %

	prd_id	prd_key	cat_key	prod_key	prd_nm	prd_cost	prd_line
1	210	CO-RF-FR-R92B-58	CO_RF	FR-R92B-58	HL Road Frame - Black- 58	NULL	R
2	211	CO-RF-FR-R92R-58	CO_RF	FR-R92R-58	HL Road Frame - Red- 58	NULL	R
3	212	AC-HE-HL-U509-R	AC_HE	HL-U509-R	Sport-100 Helmet- Red	12	S
4	213	AC-HE-HL-U509-R	AC_HE	HL-U509-R	Sport-100 Helmet- Red	14	S
5	214	AC-HE-HL-U509-R	AC_HE	HL-U509-R	Sport-100 Helmet- Red	13	S
6	215	AC-HE-HL-U509	AC_HE	HL-U509	Sport-100 Helmet- Black	12	S
7	216	AC-HE-HL-U509	AC_HE	HL-U509	Sport-100 Helmet- Black	14	S
8	217	AC-HE-HL-U509	AC_HE	HL-U509	Sport-100 Helmet- Black	13	S
9	218	CL-SO-SO-B909-M	CL_SO	SO-B909-M	Mountain Bike Socks- M	3	M

Making the SUBSTRING Function Dynamic :

- If you notice the Prod_key is off different length , so we need tot query to pick the last location number of the string by itself

- So , we use the Function **LEN()** --> Returns the number of characters in a string

```
SELECT
    prd_key,
    SUBSTRING(prd_key, 1, 5) AS cat_key,
    SUBSTRING(prd_key, 7, LEN(prd_key)) AS prod_key
FROM bronze.crm_prd_info
```

100 %

Results

Messages

	prd_key	cat_key	prod_key
1	CO-RF-FR-R92B-58	CO-RF	FR-R92B-58
2	CO-RF-FR-R92R-58	CO-RF	FR-R92R-58
3	AC-HE-HL-U509-R	AC-HE	HL-U509-R
4	AC-HE-HL-U509-R	AC-HE	HL-U509-R
5	AC-HE-HL-U509-R	AC-HE	HL-U509-R
6	AC-HE-HL-U509	AC-HE	HL-U509
7	AC-HE-HL-U509	AC-HE	HL-U509
8	AC-HE-HL-U509	AC-HE	HL-U509
9	CL-SQ-SQ-R909-M	CL-SQ	SQ-R909-M

4. Check for Unwanted Spaces in String values :

```
-- Check for Unwanted Spaces

SELECT prd_nm
FROM bronze.crm_prd_info
WHERE prd_nm != TRIM(prd_nm)
```

00 %

Results

Messages

prd_nm

5. Check the quality of numerical values :

- If there is a quantity / numerical value columns , we need to check its quality that is we need to check for any negatives or Null Values

```
SELECT prd_cost
FROM bronze.crm_prd_info
WHERE prd_cost < 0 OR prd_cost IS NULL
```

00 %

Results

Messages

	prd_cost
1	NULL
2	NULL

- Here we dont have any negative values , but we have null values . Usually we change NULL value to 0 as default
- We use the Function **ISNULL()** --> Replaces NULL Values with a specified replacement Value

- Syntax : ISNULL(col_name, value_to_replace_with)

```
SELECT prd_cost,
       ISNULL(prd_cost, 0)
FROM bronze.crm_prd_info
```

0 %

Results Messages

	prd_cost	(No column name)
1	NULL	0
2	NULL	0
3	12	12
4	14	14
5	13	13

```
SELECT prd_id,
       prd_key,
       REPLACE(SUBSTRING(prd_key, 1, 5), '-', '_') AS cat_key,
       SUBSTRING(prd_key, 7, LEN(prd_key)) AS prd_key,
       prd_nm,
       ISNULL(prd_cost, 0) AS prd_cost,
       prd_line,
       prd_start_dt,
       prd_end_dt
FROM bronze.crm_prd_info
```

100 %

Results Messages

	prd_id	prd_key	cat_key	prd_key	prd_nm	prd_cost	prd_line	prd_start_dt	
1	210	CO-RF-FR-R92B-58	CO_RF	FR-R92B-58	HL Road Frame - Black- 58	0	R	2003-07-01 00:00:00.000	p
2	211	CO-RF-FR-R92R-58	CO_RF	FR-R92R-58	HL Road Frame - Red- 58			00:00:00.000	
3	212	AC-HE-HL-U509-R	AC_HE	HL-U509-R	Sport-100 Helmet- Red	12	S	2011-07-01 00:00:00.000	:
4	213	AC-HE-HL-U509-R	AC_HE	HL-U509-R	Sport-100 Helmet- Red	14	S	2012-07-01 00:00:00.000	:
5	214	AC-HE-HL-U509-R	AC_HE	HL-U509-R	Sport-100 Helmet- Red	13	S	2013-07-01 00:00:00.000	:
6	215	AC-HE-HL-U509	AC_HE	HL-U509	Sport-100 Helmet- Black	12	S	2011-07-01 00:00:00.000	:
7	216	AC-HE-HL-U509	AC_HE	HL-U509	Sport-100 Helmet- Black	14	S	2012-07-01 00:00:00.000	:
8	217	AC-HE-HL-U509	AC_HE	HL-U509	Sport-100 Helmet- Black	13	S	2013-07-01 00:00:00.000	:
9	218	CL-SO-SO-B909-M	CL_SO	SO-B909-M	Mountain Bike Socks- M	3	M	2011-07-01 00:00:00.000	:
10	219	CL-SO-SO-B909-L	CL_SO	SO-B909-L	Mountain Bike Socks- L	3	M	2011-07-01 00:00:00.000	:
11	220	AC-HE-HL-U509-R	AC_HE	HL-U509-R	Sport-100 Helmet- Blue	12	S	2011-07-01 00:00:00.000	:

6. Need to check Data Cardinality / Standardization :

- As we can see , prd_line column has abbreviated values , we need to standardize it

```
-- Check the Cardinality & Standardize the abbreviated Values
```

```
SELECT DISTINCT prd_line
FROM bronze.crm_prd_info
```

0 %

Results Messages

prd_line
NULL
M
R
S
T

- If we don't have the exact values its better to ask and question the person who maintains the source system data
- We use the function CASE WHEN , to change the values
- **Don't Forget to put END in the closing of CASE WHEN**

```
SELECT prd_line,
CASE WHEN UPPER(TRIM(prd_line)) = 'M' THEN 'Mountain'
      WHEN UPPER(TRIM(prd_line)) = 'R' THEN 'Road'
      WHEN UPPER(TRIM(prd_line)) = 'S' THEN 'Other Sales'
      WHEN UPPER(TRIM(prd_line)) = 'T' THEN 'Touring'
      ELSE 'N/A'
END AS prd_line2
FROM bronze.crm_prd_info
```

100 %

Results Messages

	prd_line	prd_line2
1	R	Road
2	R	Road
3	S	Other Sales
4	S	Other Sales
5	S	Other Sales
6	S	Other Sales
7	S	Other Sales
8	S	Other Sales

- We can make the Functions as a common thing if we want to but if we have many conditions within we cant do this ,Like

```

SELECT prd_id,
       prd_key,
       REPLACE(SUBSTRING(prd_key, 1, 5), '-', '_') AS cat_key,
       SUBSTRING(prd_key, 7, LEN(prd_key)) AS prd_key,
       prd_nm,
       ISNULL(prd_cost, 0) AS prd_cost,
       CASE UPPER(TRIM(prd_line))
         WHEN 'M' THEN 'Mountain'
         WHEN 'R' THEN 'Road'
         WHEN 'S' THEN 'Other Sales'
         WHEN 'T' THEN 'Touring'
         ELSE 'N/A'
       END AS prd_line,
       prd_start_dt,
       prd_end_dt
FROM bronze.crm_prd_info

```

prd_id	prd_key	cat_key	prd_key	prd_nm	prd_cost	prd_line	prd_start_dt	prd_end_dt
210	CO-RF-FR-R92B-58	CO_RF	FR-R92B-58	HL Road Frame - Black- 58	0	Road	2003-07-01 00:00:00.000	NULL
211	CO-RF-FR-R92R-58	CO_RF	FR-R92R-58	HL Road Frame - Red- 58	0	Road	2003-07-01 00:00:00.000	NULL
212	AC-HE-HL-U509-R	AC_HE	HL-U509-R	Sport-100 Helmet- Red	12	Other Sales	2011-07-01 00:00:00.000	2007-12-28 00:00:00.000
213	AC-HE-HL-U509-R	AC_HE	HL-U509-R	Sport-100 Helmet- Red	14	Other Sales	2012-07-01 00:00:00.000	2008-12-27 00:00:00.000
214	AC-HE-HL-U509-R	AC_HE	HL-U509-R	Sport-100 Helmet- Red	13	Other Sales	2013-07-01 00:00:00.000	NULL
215	AC-HE-HL-U509	AC_HE	HL-U509	Sport-100 Helmet- Black	12	Other Sales	2011-07-01 00:00:00.000	2007-12-28 00:00:00.000
216	AC-HE-HL-U509	AC_HE	HL-U509	Sport-100 Helmet- Black	14	Other Sales	2012-07-01 00:00:00.000	2008-12-27 00:00:00.000
217	AC-HE-HL-U509	AC_HE	HL-U509	Sport-100 Helmet- Black	13	Other Sales	2013-07-01 00:00:00.000	NULL
218	CL-SO-SO-B909-M	CL_SO	SO-B909-M	Mountain Bike Socks- M	3	Mountain	2011-07-01 00:00:00.000	2007-12-28 00:00:00.000

7. Check for Date & Timing consistency:

- We need to check for any invalid Date / Time values , such as
 - End Time should not be less than start time , logically its not possible
 - Should not have Null Values
 - Should not have Overlapping of Date and Time between 2 or more values
 - Can have a Start Date and a Null value for End date .
 - But Should not have a Null Start Date and a value for End date
- Most Common Solution is to use the start Date of 2nd value and put it in the End date of 1st Value
- End date = Start Date of the 'Next' Record - 1** , This way the end date of 1st value is less than start of 2nd value

212	AC_HE	HL-U509-R	Sport-100 Helmet- Red	12	Other Sales	2011-07-01 00:00:00.000	2012-06-30 00:00:00.000
213	AC_HE	HL-U509-R	Sport-100 Helmet- Red	14	Other Sales	2012-07-01 00:00:00.000	2013-06-30 00:00:00.000
214	AC_HE	HL-U509-R	Sport-100 Helmet- Red	13	Other Sales	2013-07-01 00:00:00.000	NULL

```

SELECT prd_id,
       prd_key,
       prd_nm,
       prd_cost,
       prd_line,
       prd_start_dt,
       LEAD(prd_start_dt) OVER (PARTITION BY prd_key ORDER BY prd_start_dt)-1 AS prd_end_dt
FROM bronze.crm_prd_info

```

0 %

Results Messages

	prd_id	prd_key	prd_nm	prd_cost	prd_line	prd_start_dt	prd_end_dt
3	215	AC-HE-HL-U509	Sport-100 Helmet- Black	12	S	2011-07-01 00:00:00.000	2012-06-30 00:00:00.000
3	216	AC-HE-HL-U509	Sport-100 Helmet- Black	14	S	2012-07-01 00:00:00.000	2013-06-30 00:00:00.000
10	217	AC-HE-HL-U509	Sport-100 Helmet- Black	13	S	2013-07-01 00:00:00.000	NULL
11	220	AC-HE-HL-U509-B	Sport-100 Helmet- Blue	12	S	2011-07-01 00:00:00.000	2012-06-30 00:00:00.000
12	221	AC-HE-HL-U509-B	Sport-100 Helmet- Blue	14	S	2012-07-01 00:00:00.000	2013-06-30 00:00:00.000
13	222	AC-HE-HL-U509-B	Sport-100 Helmet- Blue	13	S	2013-07-01 00:00:00.000	NULL
14	212	AC-HE-HL-U509-R	Sport-100 Helmet- Red	12	S	2011-07-01 00:00:00.000	2012-06-30 00:00:00.000
15	213	AC-HE-HL-U509-R	Sport-100 Helmet- Red	14	S	2012-07-01 00:00:00.000	2013-06-30 00:00:00.000
16	214	AC-HE-HL-U509-R	Sport-100 Helmet- Red	13	S	2013-07-01 00:00:00.000	NULL
17	487	AC-HP-HY-1023-70	Hydration Pack - 70 oz.	21	S	2013-07-01 00:00:00.000	NULL
18	451	AC-I-H-T-H902	Headlights - Dual-Beam	14	R	2012-07-01 00:00:00.000	NULL

- Here we Use **LEAD() Function** --> Access values from the next row within a window

here if you notice the timing values are always 00:00:00 , it gives no meaning or use to the data so we can remove it using **CAST()**

```

SELECT prd_id,
       prd_key,
       REPLACE(SUBSTRING(prd_key, 1, 5), '-', '_') AS cat_key,
       SUBSTRING(prd_key, 7, LEN(prd_key)) AS prd_key,
       prd_nm,
       ISNULL(prd_cost, 0) AS prd_cost,
       CASE UPPER(TRIM(prd_line))
         WHEN 'M' THEN 'Mountain'
         WHEN 'R' THEN 'Road'
         WHEN 'S' THEN 'Other Sales'
         WHEN 'T' THEN 'Touring'
         ELSE 'N/A'
       END AS prd_line,
       CAST(prd_start_dt AS DATE) AS prd_start_date,
       CAST(LEAD(prd_start_dt) OVER (PARTITION BY prd_key ORDER BY prd_start_dt)-1 AS DATE) AS prd_end_dt
FROM bronze.crm_prd_info

```

100 %

Results Messages

	prd_id	prd_key	cat_key	prd_key	prd_nm	prd_cost	prd_line	prd_start_date	prd_end_dt
1	478	AC-BC-BC-M005	AC_BC	BC-M005	Mountain Bottle Cage	4	Mountain	2013-07-01	NULL
2	479	AC-BC-BC-R205	AC_BC	BC-R205	Road Bottle Cage	3	Road	2013-07-01	NULL
3	477	AC-BC-WB-H098	AC_BC	WB-H098	Water Bottle - 30 oz.	2	Other Sales	2013-07-01	NULL
4	483	AC-BR-RA-H123	AC_BR	RA-H123	Hitch Rack - 4-Bike	45	Other Sales	2013-07-01	NULL
5	486	AC-BS-ST-1401	AC_BS	ST-1401	All-Purpose Bike Stand	59	Mountain	2013-07-01	NULL
6	484	AC-CL-CL-9009	AC_CL	CL-9009	Bike Wash - Dissolver	3	Other Sales	2013-07-01	NULL
7	485	AC-FE-FE-6654	AC_FE	FE-6654	Fender Set - Mountain	8	Mountain	2013-07-01	NULL
8	215	AC-HE-HL-U509	AC_HE	HL-U509	Sport-100 Helmet- Black	12	Other Sales	2011-07-01	2012-06-30
9	216	AC-HE-HL-U509	AC_HE	HL-U509	Sport-100 Helmet- Black	14	Other Sales	2012-07-01	2013-06-30
10	217	AC-HE-HL-U509	AC_HE	HL-U509	Sport-100 Helmet- Black	13	Other Sales	2013-07-01	NULL
11	220	AC-HE-HL-U509-B	AC_HE	HL-U509-B	Sport-100 Helmet- Blue	12	Other Sales	2011-07-01	2012-06-30
12	221	AC-HE-HL-U509-B	AC_HE	HL-U509-B	Sport-100 Helmet- Blue	14	Other Sales	2012-07-01	2013-06-30
13	222	AC-HE-HL-U509-B	AC_HE	HL-U509-B	Sport-100 Helmet- Blue	13	Other Sales	2013-07-01	NULL
14	212	AC-HE-HL-U509-R	AC_HE	HL-U509-R	Sport-100 Helmet- Red	12	Other Sales	2011-07-01	2012-06-30
15	213	AC-HE-HL-U509-R	AC_HE	HL-U509-R	Sport-100 Helmet- Red	14	Other Sales	2012-07-01	2013-06-30
16	214	AC-HE-HL-U509-R	AC_HE	HL-U509-R	Sport-100 Helmet- Red	13	Other Sales	2013-07-01	NULL
17	487	AC-HP-HY-1023-70	AC_HP	HY-1023-70	Hydration Pack - 70 oz.	21	Other Sales	2013-07-01	NULL

8. Loading into Silver Layer :

- If you notice we have created new columns and did changes in datatype , then we have to go and change the DDL Query of Silver Layer and add the changes as well . These new columns are called Derived Columns

```

IF OBJECT_ID ('silver.crm_prd_info' , 'U') IS NOT NULL
    DROP TABLE silver.crm_prd_info;
-- Creating Table for Source CRM - Product Info
CREATE TABLE silver.crm_prd_info (
    prd_id          INT,
    cat_id          NVARCHAR(50),
    prd_key         NVARCHAR(50),
    prd_nm          NVARCHAR(50),
    prd_cost        INT,
    prd_line        NVARCHAR(50),
    prd_start_dt    DATE,
    prd_end_dt      DATE,
    dwh_create_date DATETIME2 DEFAULT GETDATE()
)

```


- Insert into Silver Layer and do Quality Checks

```
INSERT INTO silver.crm_prd_info(
    prd_id,
    cat_id,
    prd_key,
    prd_nm,
    prd_cost,
    prd_line,
    prd_start_dt,
    prd_end_dt
)

SELECT prd_id,
    REPLACE(SUBSTRING(prd_key, 1, 5), '-', '_') AS cat_id,
    SUBSTRING(prd_key, 7, LEN(prd_key)) AS prd_key,
    prd_nm,
    ISNULL(prd_cost, 0) AS prd_cost,
    CASE UPPER(TRIM(prd_line))
        WHEN 'M' THEN 'Mountain'
        WHEN 'R' THEN 'Road'
        WHEN 'S' THEN 'Other Sales'
        WHEN 'T' THEN 'Touring'
        ELSE 'N/A'
    END AS prd_line,
    CAST(prd_start_dt AS DATE) AS prd_start_date,
    CAST(LEAD(prd_start_dt) OVER (PARTITION BY prd_key ORDER BY prd_start_dt)-1 AS DATE) AS prd_end_dt
FROM bronze.crm_prd_info
```

```
-- Quality Checks

-- Check for Nulls or Duplicates
-- Expected Result : No Values

SELECT prd_id,
    COUNT(*)
FROM silver.crm_prd_info
GROUP BY prd_id
HAVING COUNT (*) > 1 OR prd_id IS NULL

-- Check for Unwanted Spaces
-- Expected Result : No Values

SELECT prd_nm
FROM silver.crm_prd_info
WHERE prd_nm != TRIM(prd_nm)

-- Check for Nulls or Negative Numerical Values
-- Expected Result : No Values

SELECT prd_cost
FROM silver.crm_prd_info
WHERE prd_cost < 0 OR prd_cost IS NULL

-- Check for Data Standardization & consistency

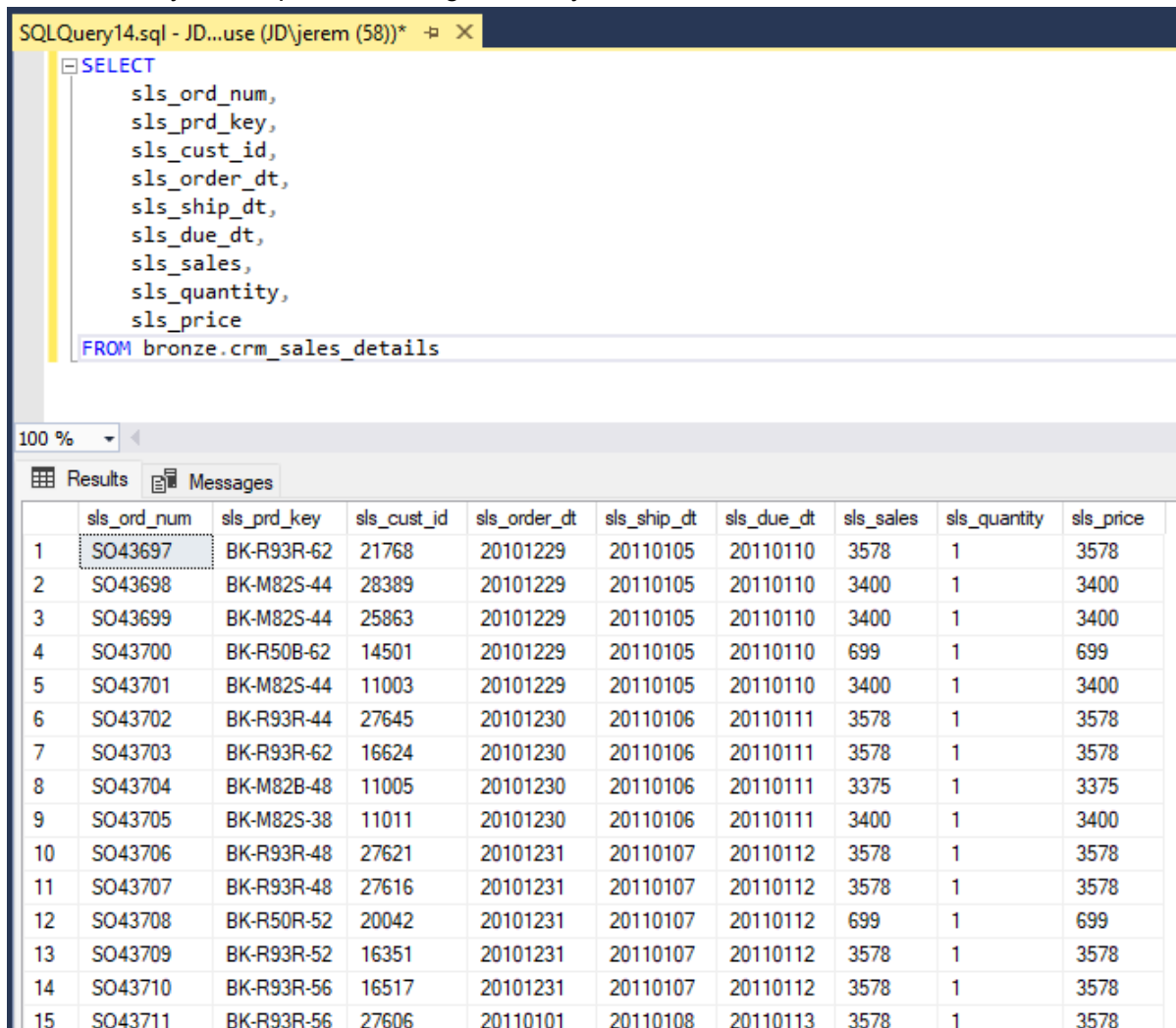
SELECT DISTINCT prd_line
FROM silver.crm_prd_info

-- Check for Invalid Date Values

SELECT *
FROM silver.crm_prd_info
WHERE prd_end_dt < prd_start_dt
```

3. crm_sales_details :

- Always initially view the table once using * then write a separate query mentioning all the column names one by one to perform changes flexibly.



The screenshot shows a SQL query window with the following query:

```
SELECT
    sls_ord_num,
    sls_prd_key,
    sls_cust_id,
    sls_order_dt,
    sls_ship_dt,
    sls_due_dt,
    sls_sales,
    sls_quantity,
    sls_price
FROM bronze.crm_sales_details
```

Below the query window, the 'Results' tab is active, displaying a table with 10 columns and 15 rows of data. The columns are: sls_ord_num, sls_prd_key, sls_cust_id, sls_order_dt, sls_ship_dt, sls_due_dt, sls_sales, sls_quantity, and sls_price. The first row is highlighted.

	sls_ord_num	sls_prd_key	sls_cust_id	sls_order_dt	sls_ship_dt	sls_due_dt	sls_sales	sls_quantity	sls_price
1	SO43697	BK-R93R-62	21768	20101229	20110105	20110110	3578	1	3578
2	SO43698	BK-M82S-44	28389	20101229	20110105	20110110	3400	1	3400
3	SO43699	BK-M82S-44	25863	20101229	20110105	20110110	3400	1	3400
4	SO43700	BK-R50B-62	14501	20101229	20110105	20110110	699	1	699
5	SO43701	BK-M82S-44	11003	20101229	20110105	20110110	3400	1	3400
6	SO43702	BK-R93R-44	27645	20101230	20110106	20110111	3578	1	3578
7	SO43703	BK-R93R-62	16624	20101230	20110106	20110111	3578	1	3578
8	SO43704	BK-M82B-48	11005	20101230	20110106	20110111	3375	1	3375
9	SO43705	BK-M82S-38	11011	20101230	20110106	20110111	3400	1	3400
10	SO43706	BK-R93R-48	27621	20101231	20110107	20110112	3578	1	3578
11	SO43707	BK-R93R-48	27616	20101231	20110107	20110112	3578	1	3578
12	SO43708	BK-R50R-52	20042	20101231	20110107	20110112	699	1	699
13	SO43709	BK-R93R-52	16351	20101231	20110107	20110112	3578	1	3578
14	SO43710	BK-R93R-56	16517	20101231	20110107	20110112	3578	1	3578
15	SO43711	BK-R93R-56	27606	20110101	20110108	20110113	3578	1	3578

1. Checking unwanted spaces :

- Here only sls_prd_key & sls_ord_num has string values so we need to check whether there is any extra unwanted spaces.

- Query = WHERE sls_ord_num != TRIM(sls_ord_num)

```

SELECT
    sls_ord_num,
    sls_prd_key,
    sls_cust_id,
    sls_order_dt,
    sls_ship_dt,
    sls_due_dt,
    sls_sales,
    sls_quantity,
    sls_price
FROM bronze.crm_sales_details
WHERE sls_ord_num != TRIM(sls_ord_num)

```

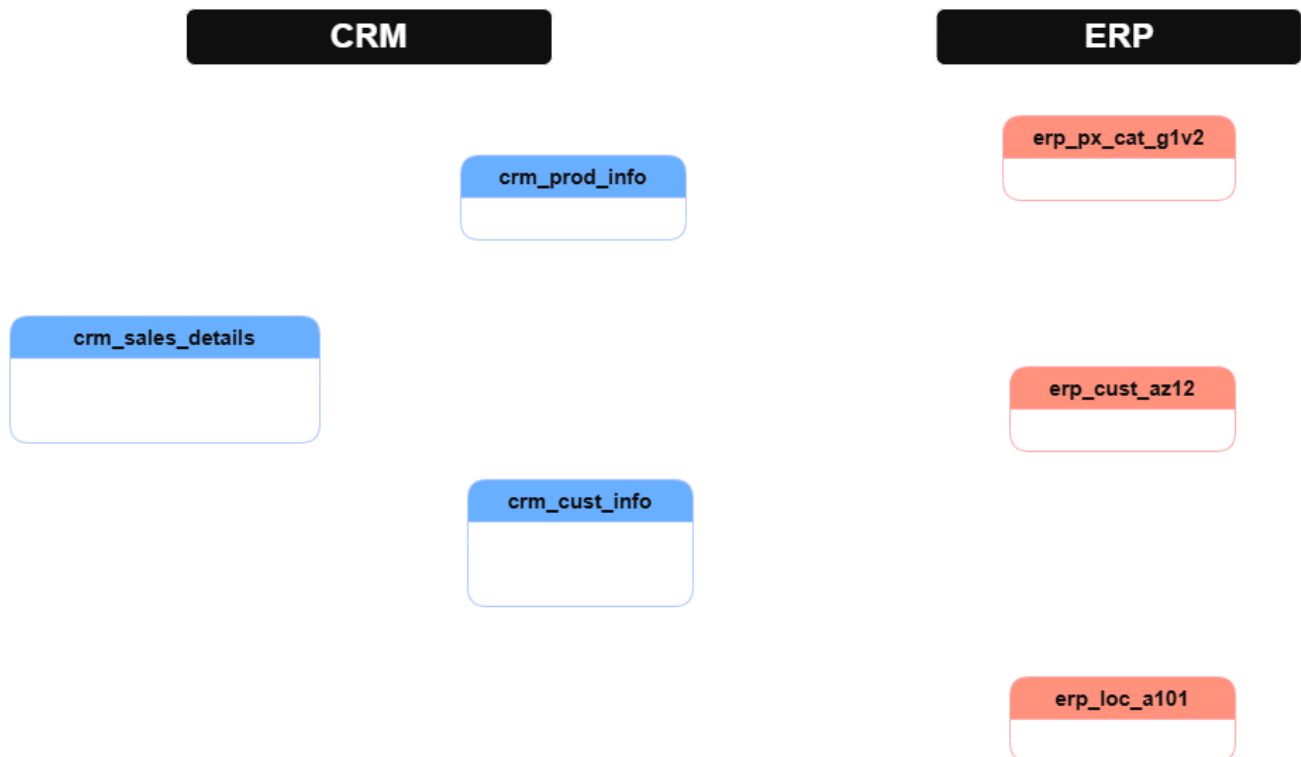
100 %

Results Messages

sls_ord_num	sls_prd_key	sls_cust_id	sls_order_dt	sls_ship_dt	sls_due_dt	sls_sales	sls_quantity	sls_price
-------------	-------------	-------------	--------------	-------------	------------	-----------	--------------	-----------

2. Checking whether any key or id missing for the connecting columns :

- As we already planned , we need to connect prd_key to crm_prod_info table & cust_id to crm_cust_info table .



- So we need to check whether all values are in the respective table as well , if any values are missing we need to check and decide whether they are important or can be removed.

```

-- Checking whether any key or id is missing from sls_prd_key & sls_cust_id so that no issue is there wh
SELECT
    sls_ord_num,
    sls_prd_key,
    sls_cust_id,
    sls_order_dt,
    sls_ship_dt,
    sls_due_dt,
    sls_sales,
    sls_quantity,
    sls_price
FROM bronze.crm_sales_details
WHERE sls_prd_key NOT IN (SELECT prd_key FROM silver.crm_prd_info)

```

sls_ord_num	sls_prd_key	sls_cust_id	sls_order_dt	sls_ship_dt	sls_due_dt	sls_sales	sls_quantity	sls_price
-------------	-------------	-------------	--------------	-------------	------------	-----------	--------------	-----------

- Here all values are in silver.crm_prod_info as well so no need to do any changes

3. Data Normalization (Converting integer to Date Values) :

- If you see here the date columns are having integer values and are not in date values

sls_cust_id	sls_order_dt	sls_ship_dt	sls_due_dt
21768	20101229	20110105	20110110
28389	20101229	20110105	20110110
25863	20101229	20110105	20110110
14501	20101229	20110105	20110110
11003	20101229	20110105	20110110
27645	20101230	20110106	20110111
16624	20101230	20110106	20110111
11005	20101230	20110106	20110111
11011	20101230	20110106	20110111
27621	20101231	20110107	20110112
27616	20101231	20110107	20110112

- We need to check any Negative values or 0 values are there are notColumn should not be <= (Less than are equal to) 0
- We'll check 1 column and will apply to all 3 columns later , Its better to check with small columns or values and once we get the solution we can apply the same to all columns

```
-- Data Normalization (Converting integer to Date Values )

SELECT
    sls_order_dt
FROM bronze.crm_sales_details
WHERE sls_order_dt <= 0
```

	sls_order_dt
1	0
2	0
3	0
4	0
5	0
6	0
7	0
8	0
9	0

- We don't have any Negative values but we have 0 , we need to clear it . Here the **best solution is replacing 0 with NULL** using **NULLIF()** --> Returns NULL if two given values are equal ; Otherwise, it returns the first expression

```
-- Data Normalization (Converting integer to Date Values )

SELECT
    NULLIF(sls_order_dt,0) sls_order_dt
FROM bronze.crm_sales_details
WHERE sls_order_dt <= 0
```

	sls_order_dt
1	NULL
2	NULL
3	NULL
4	NULL
5	NULL
6	NULL
7	NULL
8	NULL
9	NULL
10	NULL

- Then , we need to check the length of the integer , 2010/12/29 --> Year/month/Date format has totally 8 digits so there should not be any value more or less than 8 digits and we need to check it

17	NULL
18	32154
19	5489

- We have 2 values like that are less than 8 which is invalid data
- We can also check for any outliers like , a date with value greater of today that is the future and a date in a far past

```

SELECT
    NULLIF(sls_order_dt,0) sls_order_dt
FROM bronze.crm_sales_details
WHERE sls_order_dt <= 0
OR LEN(sls_order_dt) != 8
OR sls_order_dt > 20250101
OR sls_order_dt < 19990101

```

- In SQL Server, we cant change the datatype from Integer to Date Directly . We need to convert to string first then to from string to date . For this we will use Type casting using the Function **CAST()**
- Query : CASE WHEN sls_order_dt = 0 OR LEN(sls_order_dt) != 8 THEN NULL
ELSE CAST(CAST(sls_order_dt AS NVARCHAR) AS DATE)
END AS sls_order_dt,

```
-- Fixing the 0 & values that != 8 & Changeing the type of the column
```

```

SELECT
    sls_ord_num,
    sls_prd_key,
    sls_cust_id,
    CASE WHEN sls_order_dt = 0 OR LEN(sls_order_dt) != 8 THEN NULL
          ELSE CAST(CAST(sls_order_dt AS NVARCHAR) AS DATE)
    END AS sls_order_dt,

    CASE WHEN sls_ship_dt = 0 OR LEN(sls_ship_dt) != 8 THEN NULL
          ELSE CAST(CAST(sls_ship_dt AS NVARCHAR) AS DATE)
    END AS sls_ship_dt,

    CASE WHEN sls_due_dt = 0 OR LEN(sls_due_dt) != 8 THEN NULL
          ELSE CAST(CAST(sls_due_dt AS NVARCHAR) AS DATE)
    END AS sls_due_dt,
    sls_sales,
    sls_quantity,
    sls_price
FROM bronze.crm_sales_details

```

sls_ord_num	sls_prd_key	sls_cust_id	sls_order_dt	sls_ship_dt	sls_due_dt	sls_sales	sls_quantity	sls_price
SO43697	BK-R93R-62	21768	2010-12-29	2011-01-05	2011-01-10	3578	1	3578
SO43698	BK-M82S-44	28389	2010-12-29	2011-01-05	2011-01-10	3400	1	3400
SO43699	BK-M82S-44	25863	2010-12-29	2011-01-05	2011-01-10	3400	1	3400
SO43700	BK-R50B-62	14501	2010-12-29	2011-01-05	2011-01-10	699	1	699
SO43701	BK-M82S-44	11003	2010-12-29	2011-01-05	2011-01-10	3400	1	3400
SO43702	BK-R93R-44	27645	2010-12-30	2011-01-06	2011-01-11	3578	1	3578
SO43703	BK-R93R-62	16624	2010-12-30	2011-01-06	2011-01-11	3578	1	3578
SO43704	BK-M82B-48	11005	2010-12-30	2011-01-06	2011-01-11	3375	1	3375
SO43705	BK-M82S-38	11011	2010-12-30	2011-01-06	2011-01-11	3400	1	3400
SO43706	BK-R93R-48	27621	2010-12-31	2011-01-07	2011-01-12	3578	1	3578
SO43707	BK-R93R-48	27616	2010-12-31	2011-01-07	2011-01-12	3578	1	3578
SO43708	BK-R50B-62	20042	2010-12-31	2011-01-07	2011-01-12	699	1	699

- One more quality check is that the order date should not be higher than due or shipment date which is not logical

```
-- Checking Order Date > due,ship date which is wrong

SELECT *
FROM bronze.crm_sales_details
WHERE sls_order_dt > sls_ship_dt OR sls_order_dt > sls_due_dt

-- Fixing the 0 & values that != 8 & Changeing the type of the column
```

00 %

Results Messages

sls_ord_num	sls_prd_key	sls_cust_id	sls_order_dt	sls_ship_dt	sls_due_dt	sls_sales	sls_quantity	sls_price
-------------	-------------	-------------	--------------	-------------	------------	-----------	--------------	-----------

4. Data Consistency (Sales , Quantity & Price) : [Very Important]

- When it comes to Sales , Quantity & Price we have certain Business rules we need to follow :
 - ==Sales = Quantity x Price ==, (Sales Always must be equal to quantity x price)**
 - All 3 , Sales , Quantity & Price ==should not have any Negatives , Zeros , Nulls ==**

```
-----
-- Check Data Consistency (Sales,Quantity & Price)
-- >> Sales = Quantity * Price
-- >> Values should not be NULL , Zeros or Negative

SELECT
sls_sales,
sls_quantity,
sls_price
FROM bronze.crm_sales_details
WHERE sls_sales != sls_quantity * sls_price --> checking whether Sales = Quantity * Price
OR sls_sales IS NULL OR sls_quantity IS NULL OR sls_price IS NULL --> Checking whether the 3 cols have Null values
OR sls_sales <= 0 OR sls_quantity <= 0 OR sls_price <= 0 --> Checking any negative values
```

100 %

Results Messages

	sls_sales	sls_quantity	sls_price
1	10	2	NULL
2	25	5	NULL
3	70	2	NULL
4	9	1	NULL
5	35	1	NULL
6	100	10	NULL
7	16	2	NULL
8	769	1	-769
9	30	1	-30
10	22	1	-22
11	35	2	35
12	1701	1	-1701
13	21	1	-21
14	50	2	50
15	NULL	1	9
16	NULL	1	35

- We have inconsistent values . Now we have 2 ways to rectify it :
 - Solution #1 --> Talk to the source system manager , to correct it in the source system itself . Data Issue will be fixed directly in source system**
 - Solution #2 --> Sometimes the Source will be very old and we can't change in the source system so we need to perform certain calculations to Correct the Data . Data Issues has to be fixed in data warehouse**
- **There are 3 Major Rules we need to follow when fixing these data issues :**

1. **Is Sales is negative , Zero or null . We need to derive it using Quantity and Price --> $\text{Sales} = \text{Quantity} \times \text{Price}$
2. **Is Price is Zero or Null , Calculate it using Sales & Quantity --> $\text{Price} = \text{Sales} / \text{Quantity}$
3. **If Price is Negative , Convert it to a Positive Value -->

```

-----
-- Check Data Consistency (Sales,Quantity & Price)
-- >> Sales = Quantity * Price
-- >> Values should not be NULL , Zeros or Negative

SELECT DISTINCT
    sls_sales AS old_sls_sales,
    sls_quantity,
    sls_price AS old_sls_price,

    CASE WHEN sls_sales IS NULL OR sls_sales <= 0 OR sls_sales != sls_quantity * ABS(sls_price)
        THEN sls_quantity * ABS(sls_price)
        ELSE sls_sales
    END AS sls_sales,

    CASE WHEN sls_price IS NULL OR sls_price <= 0
        THEN sls_sales / NULLIF(sls_quantity, 0)
        ELSE sls_price
    END AS sls_price

FROM bronze.crm_sales_details
WHERE sls_sales != sls_quantity * sls_price --> checking whether Sales = Quantity * Price
OR sls_sales IS NULL OR sls_quantity IS NULL OR sls_price IS NULL --> Checking whether the 3 cols have Null values
OR sls_sales <= 0 OR sls_quantity <= 0 OR sls_price <= 0 --> Checking any negative values
ORDER BY sls_sales, sls_quantity , sls_price

```

	old_sls_sales	sls_quantity	old_sls_price	sls_sales	sls_price
1	NULL	1	2	2	2
2	40	1	2	2	2
3	0	1	4	4	4
4	NULL	1	8	8	8
5	9	1	NULL	9	9
6	-18	1	9	9	9
7	8	1	9	9	9
8	NULL	1	9	9	9
9	NULL	1	10	10	10
0	0	1	10	10	10
1	5	2	5	10	5

- Final Query :


```
-- Cleaning and Loading crm_sales_details
```

```
SELECT
    sls_ord_num,
    sls_prd_key,
    sls_cust_id,

    -->> Date Tables
    CASE WHEN sls_order_dt = 0 OR LEN(sls_order_dt) != 8 THEN NULL
         ELSE CAST(CAST(sls_order_dt AS NVARCHAR) AS DATE)
    END AS sls_order_dt,

    CASE WHEN sls_ship_dt = 0 OR LEN(sls_ship_dt) != 8 THEN NULL
         ELSE CAST(CAST(sls_ship_dt AS NVARCHAR) AS DATE)
    END AS sls_ship_dt,

    CASE WHEN sls_due_dt = 0 OR LEN(sls_due_dt) != 8 THEN NULL
         ELSE CAST(CAST(sls_due_dt AS NVARCHAR) AS DATE)
    END AS sls_due_dt,

    -->> Sales , Quantity & Price Tables
    CASE WHEN sls_sales IS NULL OR sls_sales <= 0 OR sls_sales != sls_quantity * ABS(sls_price)
         THEN sls_quantity * ABS(sls_price)
         ELSE sls_sales
    END AS sls_sales,

    sls_quantity,

    CASE WHEN sls_price IS NULL OR sls_price <= 0
         THEN sls_sales / NULLIF(sls_quantity, 0)
         ELSE sls_price
    END AS sls_price

FROM bronze.crm_sales_details
```

- Now we have to load this into Silver crm_sales_details . Before loading don't forget to check the DDL script and the datatypes of the column as we have changed the integer column into date we have change the DDL script as well

```
--DDL Script
```

```
IF OBJECT_ID ('silver.crm_sales_details' , 'U') IS NOT NULL
    DROP TABLE silver.crm_sales_details;
CREATE TABLE silver.crm_sales_details(
    sls_ord_num NVARCHAR(50),
    sls_prd_key NVARCHAR(50),
    sls_cust_id INT,
    sls_order_dt DATE, --> Changed
    sls_ship_dt DATE, --> Changed
    sls_due_dt DATE, --> Changed
    sls_sales INT,
    sls_quantity INT,
    sls_price INT,
    dwh_create_date DATETIME2 DEFAULT GETDATE()
)
```

-- Cleaning and Loading crm_sales_details

```
INSERT INTO silver.crm_sales_details(
    sls_ord_num,
    sls_prd_key,
    sls_cust_id,
    sls_order_dt,
    sls_ship_dt,
    sls_due_dt,
    sls_sales,
    sls_quantity,
    sls_price
)

SELECT
    sls_ord_num,
    sls_prd_key,
    sls_cust_id,

    -->> Date Tables
    CASE WHEN sls_order_dt = 0 OR LEN(sls_order_dt) != 8 THEN NULL
        ELSE CAST(CAST(sls_order_dt AS NVARCHAR) AS DATE)
    END AS sls_order_dt,

    CASE WHEN sls_ship_dt = 0 OR LEN(sls_ship_dt) != 8 THEN NULL
        ELSE CAST(CAST(sls_ship_dt AS NVARCHAR) AS DATE)
    END AS sls_ship_dt,

    CASE WHEN sls_due_dt = 0 OR LEN(sls_due_dt) != 8 THEN NULL
        ELSE CAST(CAST(sls_due_dt AS NVARCHAR) AS DATE)
    END AS sls_due_dt,

    -->> Sales , Quantity & Price Tables
    CASE WHEN sls_sales IS NULL OR sls_sales <= 0 OR sls_sales != sls_quantity * ABS(sls_price)
        THEN sls_quantity * ABS(sls_price)
        ELSE sls_sales
    END AS sls_sales,

    sls_quantity,

    CASE WHEN sls_price IS NULL OR sls_price <= 0
        THEN sls_sales / NULLIF(sls_quantity, 0)
        ELSE sls_price
    END AS sls_price

FROM bronze.crm_sales_details
```

100 %

Messages

(60398 rows affected)

ERP TABLES :

1. erp_cust_az12 :

SQLQuery2.sql - JD\...use (JD\jerem (51))*

```

SELECT
  cid,
  bdate,
  gen
FROM bronze.erp_cust_az12

```

100 %

Results Messages

	cid	bdate	gen
1	NASAW00011000	1971-10-06	Male
2	NASAW00011001	1976-05-10	Male
3	NASAW00011002	1971-02-09	Male
4	NASAW00011003	1973-08-14	Female
5	NASAW00011004	1979-08-05	Female
6	NASAW00011005	1976-08-01	Male
7	NASAW00011006	1976-12-02	Female
8	NASAW00011007	1969-11-06	Male
9	NASAW00011008	1975-07-04	Female
10	NASAW00011009	1969-09-29	Male
11	NASAW00011010	1969-08-05	Female
12	NASAW00011011	1969-05-03	Male

- We can check the data design and understand that the cid can be connected to cst_key in crm_cust_info

```

SELECT
    cid,
    bdate,
    gen
FROM bronze.erp_cust_az12

SELECT * FROM [silver].[crm_cust_info]

```

100 %

Results Messages

	cid	bdate	gen
1	NASAW00011000	1971-10-06	Male
2	NASAW00011001	1976-05-10	Male
3	NASAW00011002	1971-02-09	Male
4	NASAW00011003	1973-08-14	Female
5	NASAW00011004	1979-08-05	Female
6	NASAW00011005	1976-08-01	Male
7	NASAW00011006	1976-12-02	Female
8	NASAW00011007	1969-11-06	Male
9	NASAW00011008	1975-07-04	Female
10	NASAW00011009	1969-09-29	Male
11	NASAW00011010	1969-08-05	Female
12	NASAW00011011	1969-05-03	Male
13	NASAW00011012	1979-01-14	Female

	cst_id	cst_key	cst_firstname	cst_lastname	cst_marital_status	cst_gndr	cst_create_date	dwh_create_date
1	11000	AW00011000	Jon	Yang	Married	Male	2025-10-06	2026-03-12 10:01:48.0366667
2	11001	AW00011001	Eugene	Huang	Single	Male	2025-10-06	2026-03-12 10:01:48.0366667
3	11002	AW00011002	Ruben	Torres	Married	Male	2025-10-06	2026-03-12 10:01:48.0366667
4	11003	AW00011003	Christy	Zhu	Single	Female	2025-10-06	2026-03-12 10:01:48.0366667
5	11004	AW00011004	Elizabeth	Johnson	Single	Female	2025-10-06	2026-03-12 10:01:48.0366667
6	11005	AW00011005	Julio	Ruiz	Single	Male	2025-10-06	2026-03-12 10:01:48.0366667
7	11006	AW00011006	Janet	Alvarez	Single	Female	2025-10-06	2026-03-12 10:01:48.0366667

--> Checking the extra String in key :

- we can notice that in cst_key we dont have the 1st 3 Letters of Cid of erp table , thus we can remove those 3 letter

- We can use the **==SUBSTRING() Function ==** to remove the NAS and keep the remaining string

```

SELECT
  cid,
  CASE WHEN cid LIKE 'NAS%' THEN SUBSTRING(cid, 4, LEN(cid))
  ELSE cid
END AS cid,
  bdate,
  gen
FROM bronze.erp_cust_az12

```

100 %

Results Messages

	cid	cid	bdate	gen
1	NASAW00011000	AW00011000	1971-10-06	Male
2	NASAW00011001	AW00011001	1976-05-10	Male
3	NASAW00011002	AW00011002	1971-02-09	Male
4	NASAW00011003	AW00011003	1973-08-14	Female
5	NASAW00011004	AW00011004	1979-08-05	Female
6	NASAW00011005	AW00011005	1976-08-01	Male
7	NASAW00011006	AW00011006	1976-12-02	Female
8	NASAW00011007	AW00011007	1969-11-06	Male
9	NASAW00011008	AW00011008	1975-07-04	Female

--> Checking the Out-of-Range Dates

- We need to check whether any date is far more in the past or in the future that is more than the current date

```

SELECT DISTINCT
  bdate
FROM bronze.erp_cust_az12
WHERE bdate < '1924-01-01' OR bdate > GETDATE()

```

0 %

Results Messages

	bdate
1	1916-02-10
2	1917-02-09
3	1917-06-05
4	1917-09-20
5	1918-02-11
5	1918-11-08
7	1919-02-14
3	1919-03-10
3	1920-11-14
10	1922-01-02
11	1922-04-10
12	1922-06-06

- We can correct this by replacing the wrong values with NULL

```
SELECT
    CASE WHEN cid LIKE 'NAS%' THEN SUBSTRING(cid, 4, LEN(cid))
    ELSE cid
END AS cid,
    bdate,
    CASE WHEN bdate > GETDATE() THEN NULL
    ELSE bdate
END AS bdate,
    gen
FROM bronze.erp_cust_az12
```

00 %

Results Messages

	cid	bdate	bdate	gen
2...	AW00013417	2050-09-07	NULL	Male
2...	AW00013418	1970-12-01	1970-12-01	Male
2...	AW00013419	1970-12-21	1970-12-21	Female
2...	AW00013420	1971-01-30	1971-01-30	Female
2...	AW00013421	1970-12-21	1970-12-21	Female

--> Checking the Data standardization & consistency :

```
-- Checking the Data standardization & consistency :
SELECT DISTINCT gen
FROM bronze.erp_cust_az12
```

00 %

Results Messages

	gen
1	NULL
2	F
3	
4	Male
5	Female
6	M

```
-- Correcting Data Consistnecy
```

```
SELECT
    CASE WHEN cid LIKE 'NAS%' THEN SUBSTRING(cid, 4, LEN(cid))
    ELSE cid
    END AS cid,
    CASE WHEN bdate > GETDATE() THEN NULL
    ELSE bdate
    END AS bdate,
    CASE WHEN UPPER(TRIM(gen)) IN ('F', 'FEMALE') THEN 'Female'
    WHEN UPPER(TRIM(gen)) IN ('M', 'MALE') THEN 'Male'
    ELSE 'N/A'
    END as gen
FROM bronze.erp_cust_az12
```

cid	bdate	gen
AW00011000	1971-10-06	Male
AW00011001	1976-05-10	Male
AW00011002	1971-02-09	Male
AW00011003	1973-08-14	Female
AW00011004	1979-08-05	Female
AW00011005	1976-08-01	Male
AW00011006	1976-12-02	Female
AW00011007	1969-11-06	Male
AW00011008	1975-07-04	Female
AW00011009	1969-09-29	Male
AW00011010	1969-08-05	Female
AW00011011	1969-05-03	Male

- With that we have cleaned the customer erp table . Now insert these values into silver schema

```
-- INSERTING INTO SILVER LAYER
INSERT INTO silver.erp_cust_az12(
    cid,
    bdate,
    gen
)
```

- Now do the quality checks in the silver layer

2. erp_loc_a101 :

```

SELECT
    cid,
    cntry
FROM bronze.erp_loc_a101

SELECT cst_key FROM [silver].crm_cust_info

```

100 %

Results Messages

	cid	cntry
1	AW-00011000	Australia
2	AW-00011001	Australia
3	AW-00011002	Australia
4	AW-00011003	Australia
5	AW-00011004	Australia
6	AW-00011005	Australia
7	AW-00011006	Australia
8	AW-00011007	Australia
9	AW-00011008	Australia

	cst_key
1	AW00011000
2	AW00011001
3	AW00011002
4	AW00011003
5	AW00011004
6	AW00011005
7	AW00011006

- If we compare and see here in erp table we have a '-' in-between the string and we need to remove it , we can do that by **REPLACE() Function**

```

SELECT
    REPLACE(cid, '-', '') cid,
    cntry
FROM bronze.erp_loc_a101

SELECT cst_key FROM [silver].crm_cust_info

```

100 %

Results Messages

	cid	cntry
1	AW00011000	Australia
2	AW00011001	Australia
3	AW00011002	Australia
4	AW00011003	Australia
5	AW00011004	Australia

	cst_key
1	AW00011000
2	AW00011001
3	AW00011002
4	AW00011003
5	AW00011004
6	AW00011005

- Now its matching

--> **Checking Data consistency**


```
-- Checking Data Consistency
```

```
SELECT DISTINCT cntry  
FROM bronze.erp_loc_a101  
ORDER BY cntry
```

100 %

Results Messages

	cntry
1	NULL
2	
3	Australia
4	Canada
5	DE
6	France
7	Germany
8	United Kingdom
9	United States
10	US
11	USA

```
SELECT  
  REPLACE(cid, '-', '') cid,  
  cntry,  
  CASE WHEN TRIM(cntry) = 'DE' THEN 'Germany'  
        WHEN TRIM(cntry) IN ('US', 'USA') THEN 'United States'  
        WHEN TRIM(cntry) = '' OR cntry IS NULL THEN 'N/A'  
        ELSE TRIM(cntry)  
  END AS cntry  
FROM bronze.erp_loc_a101
```

0 %

Results Messages

	cid	cntry	cntry
...	AW00018577	USA	United States
...	AW00018578	USA	United States
...	AW00018579	USA	United States
...	AW00018580	Canada	Canada
...	AW00018581	USA	United States
...	AW00018582	USA	United States
...	AW00018583	USA	United States
...	AW00018584	USA	United States
...	AW00018585	USA	United States
...	AW00018586	USA	United States
...	AW00018587	USA	United States
...	AW00018588	USA	United States
...	AW00018589	USA	United States

- Insert into Silver Layer

```
-- Insert into silver Layer

INSERT INTO silver.erp_loc_a101 (
    cid,
    cntry
)

SELECT
    REPLACE(cid, '-', '') cid,
    CASE WHEN TRIM(cntry) = 'DE' THEN 'Germany'
         WHEN TRIM(cntry) IN ('US', 'USA') THEN 'United States'
         WHEN TRIM(cntry) = '' OR cntry IS NULL THEN 'N/A'
         ELSE TRIM(cntry)
    END AS cntry
FROM bronze.erp_loc_a101
```

%

Messages

(18484 rows affected)

Completion time: 2026-03-16T11:43:18.8772834+05:30

3. erp_px_cat_g1v2 :

```
SELECT
    id,
    cat,
    subcat,
    maintenance
FROM bronze.erp_px_cat_g1v2

-- Check for unwanted spaces
SELECT * FROM bronze.erp_px_cat_g1v2
WHERE cat != TRIM(cat) OR subcat != TRIM(subcat) OR maintenance != TRIM(maintenance)

-- Check for Data Consistency

SELECT DISTINCT
    cat
FROM bronze.erp_px_cat_g1v2

SELECT DISTINCT
    subcat
FROM bronze.erp_px_cat_g1v2

SELECT DISTINCT
    maintenance
FROM bronze.erp_px_cat_g1v2
```

Create a Stored procedure for Silver Layer

- Beautify and create a stored procedure with try and catch to record the timing

```

EXEC silver_load_silver

-- CREATING STORED PROCEDURE
CREATE OR ALTER PROCEDURE silver_load_silver AS
BEGIN

    DECLARE @start_time DATETIME, @end_time DATETIME, @batch_start_time DATETIME, @batch_end_time DATETIME;
    BEGIN TRY

        SET @batch_start_time = GETDATE();

        PRINT '=====';
        PRINT '-----LOADING SILVER LAYER-----';
        PRINT '=====';

        PRINT '*****';
        PRINT 'Loading CRM Tables';
        PRINT '*****';

        PRINT '-----';
        -- TRUNCATING AND INSERTING FOR AVOIDING DUPLICATION OF DATA

        -- Silver CRM Customer Info

        SET @start_time = GETDATE();
        PRINT '>>> Truncating Table : silver.crm_cust_info';
        TRUNCATE TABLE silver.crm_cust_info;

        PRINT '>>> Inserting Data Into : silver.crm_cust_info';
        INSERT INTO silver.crm_cust_info (
            cst_id,
            cst_key,
            cst_firstname,
            cst_lastname,
            cst_marital_status,
            cst_gndr,
            . . .

            -----,
            maintenance
        )

        SELECT
            id,
            cat,
            subcat,
            maintenance
        FROM bronze.erp_px_cat_glv2

        SET @end_time = GETDATE();
        PRINT '>> LOAD DURATION : ' + CAST(DATEDIFF(second, @start_time, @end_time) AS NVARCHAR) + ' seconds'

        PRINT '-----';

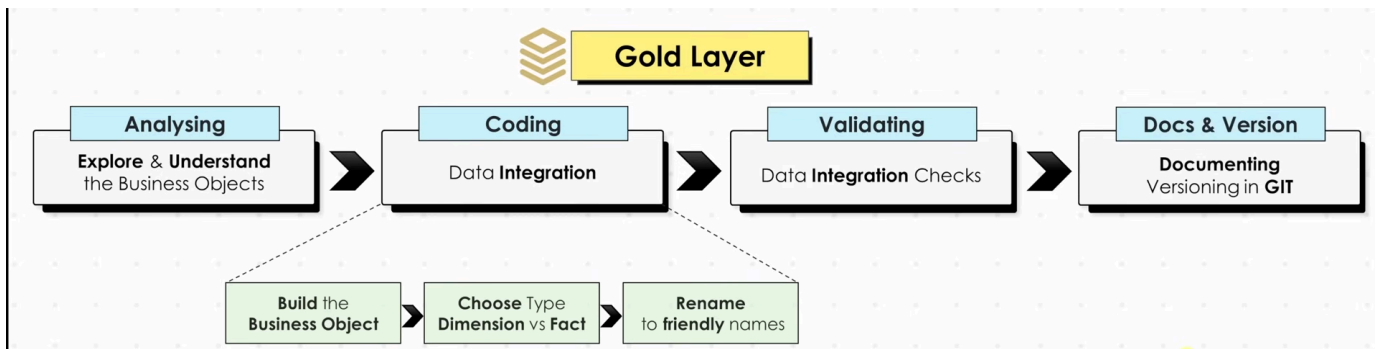
        SET @batch_end_time = GETDATE();
        PRINT '=====';
        PRINT '>> LOADING DATA INTO SILVER LAYER COMPLETED'
        PRINT '>> TOTAL LOAD DURATION OF SILVER LAYER: ' + CAST(DATEDIFF(second, @batch_start_time, @batch_end_time) AS NVARCHAR) + ' seconds';
        PRINT '=====';

    END TRY

    BEGIN CATCH
        PRINT '*****';
        PRINT 'ERROR OCCURED DURING LOADING SILVER LAYER';
        PRINT 'Error Message' + ERROR_MESSAGE();
        PRINT 'Error Message' + CAST (ERROR_NUMBER() AS NVARCHAR);
        PRINT 'Error Message' + CAST (ERROR_STATE() AS NVARCHAR);
        PRINT '*****';
    END CATCH
END

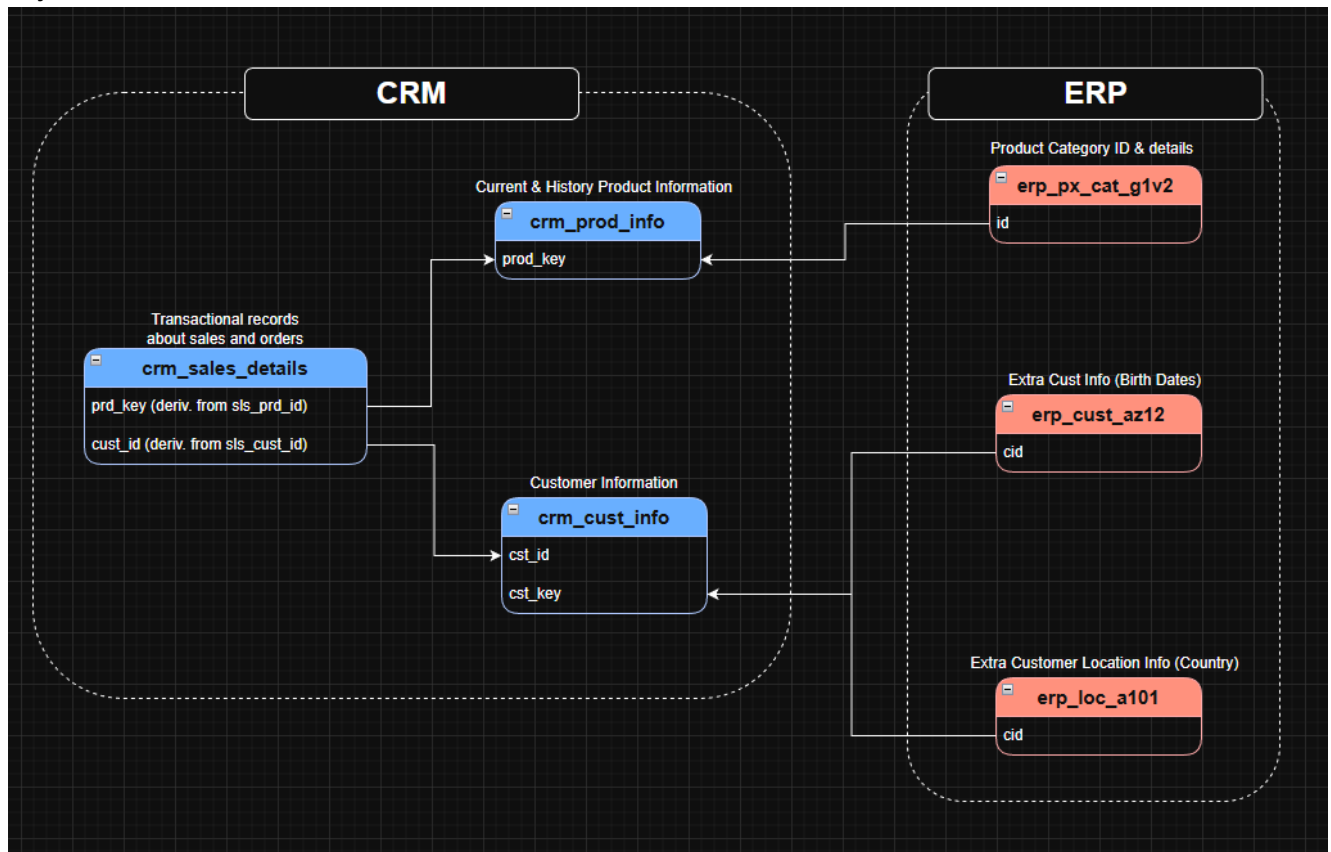
```

4. Building Gold Layer :

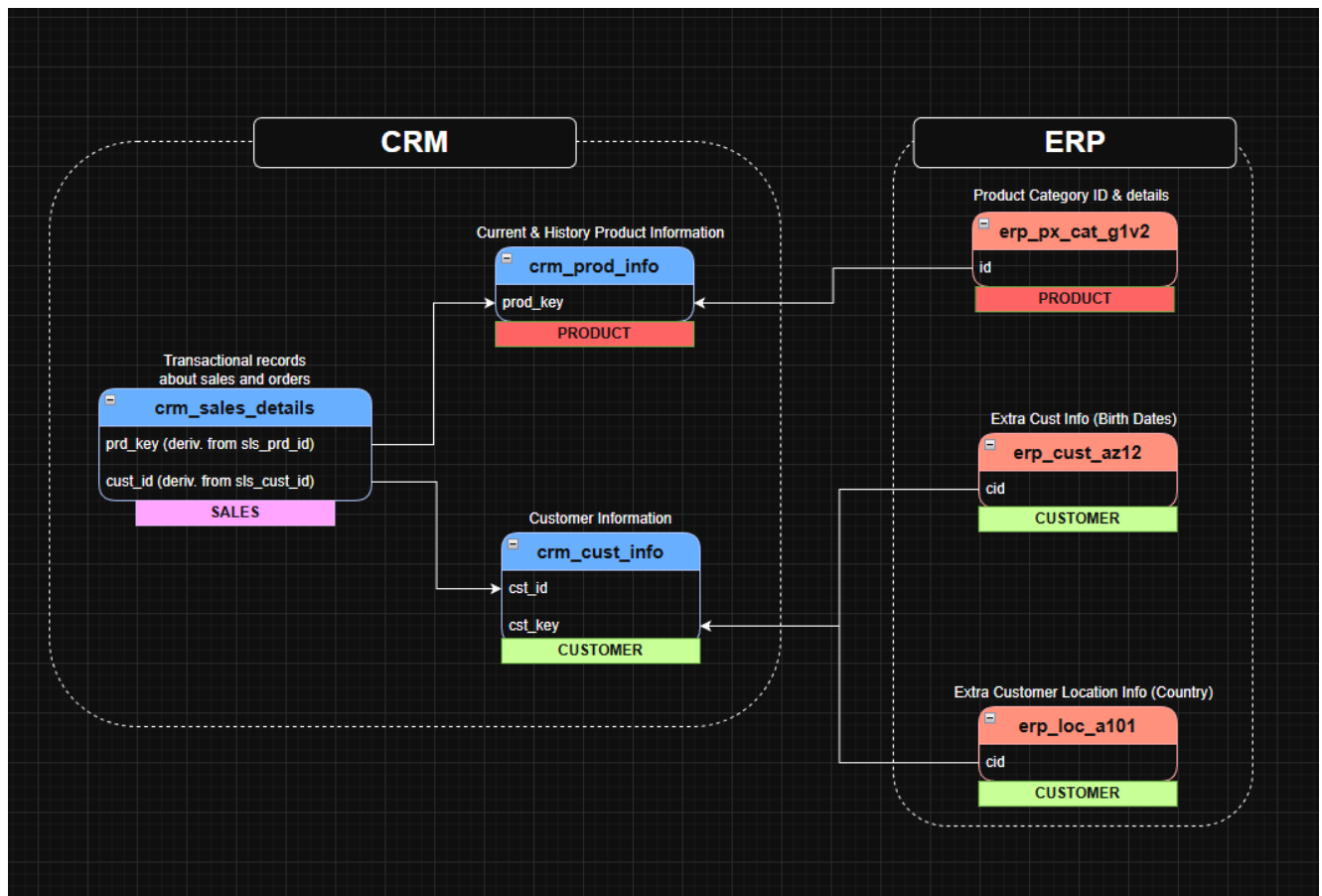


1. Analyzing Business Logics :

- In order to understand the Business model , we have to understand the original initial model on how the objects /tables are related to each other



- First , add a tag to the existing model diagram for telling which tables are related to what ?



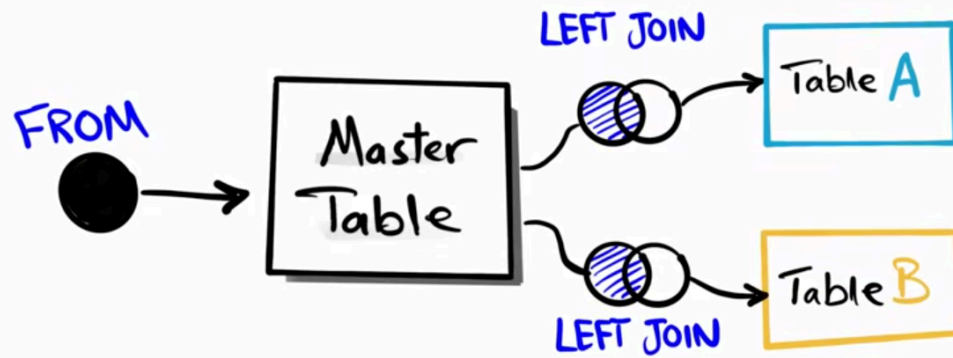
2. Data Integration :

- This time we wont be creating tables but will be building views
- We will be doing only data transformations like :
 1. Data Integration
 2. Data Aggregation

Dimension Table - Customer

<pre> SELECT ci.cst_id, ci.cst_key, ci.cst_firstname, ci.cst_lastname, ci.cst_marital_status, ci.cst_gndr, ci.cst_create_date FROM silver.crm_cust_info ci -- ci --> is alias for the table </pre>							
100 %							
Results Messages							
	cst_id	cst_key	cst_firstname	cst_lastname	cst_marital_status	cst_gndr	cst_create_date
1	11000	AW00011000	Jon	Yang	Married	Male	2025-10-06
2	11001	AW00011001	Eugene	Huang	Single	Male	2025-10-06
3	11002	AW00011002	Ruben	Torres	Married	Male	2025-10-06
4	11003	AW00011003	Christy	Zhu	Single	Female	2025-10-06
5	11004	AW00011004	Elizabeth	Johnson	Single	Female	2025-10-06
6	11005	AW00011005	Julio	Ruiz	Single	Male	2025-10-06
7	11006	AW00011006	Janet	Alvarez	Single	Female	2025-10-06
8	11007	AW00011007	Marco	Mehta	Married	Male	2025-10-06
9	11008	AW00011008	Rob	Verhoff	Single	Female	2025-10-06
10	11009	AW00011009	Shannon	Carlson	Single	Male	2025-10-06
11	11010	AW00011010	Jacquelyn	Suarez	Single	Female	2025-10-06
12	11011	AW00011011	Curtis	Lu	Married	Male	2025-10-06
13	11012	AW00011012	Lauren	Walker	Married	Female	2025-10-06
14	11013	AW00011013	Ian	Jenkins	Married	Male	2025-10-06

- Since this is a customer table as per the data model diagram we need to connect this with erp_cust_az12 & erp_loc_a101
- **While Doing Joins Always remember to start with the master table and try to avoid inner join because the other source might not have all the data . When you do inner join it just appends to the master table with the matching values without touching the master table**



My Joining Strategy

Result

Master							A			B	
m	n	~	~	~	~	~	~	~	~	~	~
~	~	~	~	~	~	~	~	~	~	~	~
~	~	~	~	~	~	~	~	~	~	~	~
~	~	~	~	~	~	~	~	~	~	~	~
~	~	~	~	~	~	~	~	~	~	~	~
~	~	~	~	~	~	~	~	~	~	~	~
~	~	~	~	~	~	~	~	~	~	~	~
~	~	~	~	~	~	~	~	~	~	~	~

```

SELECT
  ci.cst_id,
  ci.cst_key,
  ci.cst_firstname,
  ci.cst_lastname,
  ci.cst_marital_status,
  ci.cst_gndr,
  ci.cst_create_date,
  ca.bdate,
  ca.gen
FROM silver.crm_cust_info ci -- ci --> is alias for the table
LEFT JOIN silver.erp_cust_az12 ca
ON      ci.cst_key = ca.cid
  
```

100 %

Results Messages

	cst_id	cst_key	cst_firstname	cst_lastname	cst_marital_status	cst_gndr	cst_create_date	bdate	gen
1	11000	AW00011000	Jon	Yang	Married	Male	2025-10-06	1971-10-06	Male
2	11001	AW00011001	Eugene	Huang	Single	Male	2025-10-06	1976-05-10	Male
3	11002	AW00011002	Ruben	Torres	Married	Male	2025-10-06	1971-02-09	Male
4	11003	AW00011003	Christy	Zhu	Single	Female	2025-10-06	1973-08-14	Female
5	11004	AW00011004	Elizabeth	Johnson	Single	Female	2025-10-06	1979-08-05	Female
6	11005	AW00011005	Julio	Ruiz	Single	Male	2025-10-06	1976-08-01	Male
7	11006	AW00011006	Janet	Alvarez	Single	Female	2025-10-06	1976-12-02	Female
8	11007	AW00011007	Marco	Mehta	Married	Male	2025-10-06	1969-11-06	Male
9	11008	AW00011008	Rob	Verhoff	Single	Female	2025-10-06	1975-07-04	Female
10	11009	AW00011009	Shannon	Carlson	Single	Male	2025-10-06	1969-09-29	Male
11	11010	AW00011010	Jacquelyn	Suarez	Single	Female	2025-10-06	1969-08-05	Female
12	11011	AW00011011	Curtis	Lu	Married	Male	2025-10-06	1969-05-03	Male

- Don't forget to mention the required column in SELECT query list (ca.gen, ca.bdate)

```

SELECT
    ci.cst_id,
    ci.cst_key,
    ci.cst_firstname,
    ci.cst_lastname,
    ci.cst_marital_status,
    ci.cst_gndr,
    ci.cst_create_date,
    ca.bdate,
    ca.gen,
    la.cntry
FROM silver.crm_cust_info ci -- ci --> is alias for the table
LEFT JOIN silver.erp_cust_az12 ca
ON      ci.cst_key = ca.cid
LEFT JOIN silver.erp_loc_a101 la
ON      ci.cst_key = la.cid

```

100 %

Results Messages

	cst_id	cst_key	cst_firstname	cst_lastname	cst_marital_status	cst_gndr	cst_create_date	bdate	gen	cntry
1	11000	AW00011000	Jon	Yang	Married	Male	2025-10-06	1971-10-06	Male	Australia
2	11001	AW00011001	Eugene	Huang	Single	Male	2025-10-06	1976-05-10	Male	Australia
3	11002	AW00011002	Ruben	Torres	Married	Male	2025-10-06	1971-02-09	Male	Australia
4	11003	AW00011003	Christy	Zhu	Single	Female	2025-10-06	1973-08-14	Female	Australia
5	11004	AW00011004	Elizabeth	Johnson	Single	Female	2025-10-06	1979-08-05	Female	Australia
6	11005	AW00011005	Julio	Ruiz	Single	Male	2025-10-06	1976-08-01	Male	Australia
7	11006	AW00011006	Janet	Alvarez	Single	Female	2025-10-06	1976-12-02	Female	Australia
8	11007	AW00011007	Marco	Mehta	Married	Male	2025-10-06	1969-11-06	Male	Australia
9	11008	AW00011008	Rob	Verhoff	Single	Female	2025-10-06	1975-07-04	Female	Australia
10	11009	AW00011009	Shannon	Carlson	Single	Male	2025-10-06	1969-09-29	Male	Australia
11	11010	AW00011010	Jacquelyn	Suarez	Single	Female	2025-10-06	1969-08-05	Female	Australia

Always Check is any duplicates were introduced by the Join Logic

- For that we use **GROUP BY** function


```

SELECT cst_id, COUNT(*) FROM
(
  SELECT
    ci.cst_id,
    ci.cst_key,
    ci.cst_firstname,
    ci.cst_lastname,
    ci.cst_marital_status,
    ci.cst_gndr,
    ci.cst_create_date,
    ca.bdate,
    ca.gen,
    la.cntry
  FROM silver.crm_cust_info ci -- ci --> is alias for the table
  LEFT JOIN silver.erp_cust_az12 ca
  ON      ci.cst_key = ca.cid
  LEFT JOIN silver.erp_loc_a101 la
  ON      ci.cst_key = la.cid)t

GROUP BY cst_id
HAVING COUNT(*) > 1

```

100 %

Results Messages

cst_id	(No column name)
--------	------------------

--> Now Check if any columns are having same identical values , Here we have two Gender Columns

- Using DISTINCT & ORDER BY , Check the unique values of the columns

```

SELECT DISTINCT
  ci.cst_gndr,
  ca.gen
FROM silver.crm_cust_info ci -- ci --> is alias for the table
LEFT JOIN silver.erp_cust_az12 ca
ON      ci.cst_key = ca.cid
LEFT JOIN silver.erp_loc_a101 la
ON      ci.cst_key = la.cid
ORDER BY 1,2

```

100 %

Results Messages

	cst_gndr	gen
1	Female	Female
2	Female	Male
3	Female	N/A
4	Male	Female
5	Male	Male
6	Male	N/A
7	N/A	NULL
8	N/A	Female
9	N/A	Male
10	N/A	N/A

- Here we can see that some values are mismatching like in cst_gndr column the value is Female and in gen it is Male . This is a data inconsistency and we need to correct it.

- And the NULL value is because there is not matching value in Master table
- To solve this we can either ask the Source system manager or the most common way is to take the value from Master Source , Here CRM Data is the Master Source of Customer data so we'll take that value into consideration

```

SELECT DISTINCT
    ci.cst_gndr,
    ca.gen,
    CASE WHEN ci.cst_gndr != 'n/a' THEN ci.cst_gndr -- CRM is the Master for Gender Info
    ELSE COALESCE(ca.gen, 'n/a')
    END AS new_gen
FROM silver.crm_cust_info ci -- ci --> is alias for the table
LEFT JOIN silver.erp_cust_az12 ca
ON      ci.cst_key = ca.cid
LEFT JOIN silver.erp_loc_a101 la
ON      ci.cst_key = la.cid
ORDER BY 1,2

```

100 %

Results Messages

	cst_gndr	gen	new_gen
1	Female	Female	Female
2	Female	Male	Female
3	Female	N/A	Female
4	Male	Female	Male
5	Male	Male	Male
6	Male	N/A	Male
7	N/A	NULL	n/a
8	N/A	Female	Female
9	N/A	Male	Male
10	N/A	N/A	N/A

Renaming the columns into Understandable names

- Using **AS query** rename each column into meaningful column names

```

SELECT
    ci.cst_id AS customer_id,
    ci.cst_key AS customer_number,
    ci.cst_firstname AS first_name,
    ci.cst_lastname AS last_name,
    ci.cst_marital_status AS marital_status,
    CASE WHEN ci.cst_gndr != 'n/a' THEN ci.cst_gndr -- CRM is the Master for Gender Info
    ELSE COALESCE(ca.gen, 'n/a')
    END AS gender,
    ci.cst_create_date AS create_date,
    ca.bdate AS birthdate,
    la.cntry AS country
FROM silver.crm_cust_info ci -- ci --> is alias for the table
LEFT JOIN silver.erp_cust_az12 ca
ON      ci.cst_key = ca.cid
LEFT JOIN silver.erp_loc_a101 la
ON      ci.cst_key = la.cid

```

100 %

Results Messages

	customer_id	customer_number	first_name	last_name	marital_status	gender	create_date	birthdate	country
1	11000	AW00011000	Jon	Yang	Married	Male	2025-10-06	1971-10-06	Australia
2	11001	AW00011001	Eugene	Huang	Single	Male	2025-10-06	1976-05-10	Australia
3	11002	AW00011002	Ruben	Torres	Married	Male	2025-10-06	1971-02-09	Australia
4	11003	AW00011003	Christy	Zhu	Single	Female	2025-10-06	1973-08-14	Australia
5	11004	AW00011004	Elizabeth	Johnson	Single	Female	2025-10-06	1979-08-05	Australia

Sort the Columns into Logical Groups :

- Reorder the columns in more logical ordered columns

```

SELECT
    ci.cst_id AS customer_id,
    ci.cst_key AS customer_number,
    ci.cst_firstname AS first_name,
    ci.cst_lastname AS last_name,
    la.cntry AS country,
    ci.cst_marital_status AS marital_status,
    CASE WHEN ci.cst_gndr != 'n/a' THEN ci.cst_gndr -- CRM is the Master for Gender Info
    ELSE COALESCE(ca.gen, 'n/a')
    END AS gender,
    ca.bdate AS birthdate,
    ci.cst_create_date AS create_date
FROM silver.crm_cust_info ci -- ci --> is alias for the table
LEFT JOIN silver.erp_cust_az12 ca
ON      ci.cst_key = ca.cid
LEFT JOIN silver.erp_loc_a101 la
ON      ci.cst_key = la.cid

```

100 %

Results Messages

	customer_id	customer_number	first_name	last_name	country	marital_status	gender	birthdate	create_date
1	11000	AW00011000	Jon	Yang	Australia	Married	Male	1971-10-06	2025-10-06
2	11001	AW00011001	Eugene	Huang	Australia	Single	Male	1976-05-10	2025-10-06
3	11002	AW00011002	Ruben	Torres	Australia	Married	Male	1971-02-09	2025-10-06
4	11003	AW00011003	Christy	Zhu	Australia	Single	Female	1973-08-14	2025-10-06
5	11004	AW00011004	Elizabeth	Johnson	Australia	Single	Female	1979-08-05	2025-10-06

Decide whether it is a Dimensional or Fact Table :

- Since this table describes and gives more information about customers and not giving any values about the transaction happening by the customers . This is a Dimensional Table
- Mostly in Data Warehouse , **we will create new primary keys called Surrogate Key**
- **Surrogate Key is a System Generated unique identifier assigned to each record in a table** , It has no business meaning or usage. There are different way of generating a surrogate keys :
 1. We can Define it in DDL-based Generation
 2. Query - based using Window Function(Row_number)

```

SELECT
  ROW_NUMBER() OVER (ORDER BY cst_id) AS customer_key,
  ci.cst_id AS customer_id,
  ci.cst_key AS customer_number,
  ci.cst_firstname AS first_name,
  ci.cst_lastname AS last_name,
  la.cntry AS country,
  ci.cst_marital_status AS marital_status,
  CASE WHEN ci.cst_gndr != 'n/a' THEN ci.cst_gndr -- CRM is the Master for Gender Info
        ELSE COALESCE(ca.gen, 'n/a')
  END AS gender,
  ca.bdate AS birthdate,
  ci.cst_create_date AS create_date
FROM silver.crm_cust_info ci -- ci --> is alias for the table
LEFT JOIN silver.erp_cust_az12 ca
ON
  ci.cst_key = ca.cid
LEFT JOIN silver.erp_loc_a101 la
ON
  ci.cst_key = la.cid

```

100 %

Results Messages

	customer_key	customer_id	customer_number	first_name	last_name	country	marital_status	gender	birthdate	create_date
1	1	11000	AW00011000	Jon	Yang	Australia	Married	Male	1971-10-06	2025-10-06
2	2	11001	AW00011001	Eugene	Huang	Australia	Single	Male	1976-05-10	2025-10-06
3	3	11002	AW00011002	Ruben	Torres	Australia	Married	Male	1971-02-09	2025-10-06
4	4	11003	AW00011003	Christy	Zhu	Australia	Single	Female	1973-08-14	2025-10-06
5	5	11004	AW00011004	Elizabeth	Johnson	Australia	Single	Female	1979-08-05	2025-10-06
6	6	11005	AW00011005	Julio	Ruiz	Australia	Single	Male	1976-08-01	2025-10-06
7	7	11006	AW00011006	Isabel	Alvarez	Australia	Single	Female	1976-12-02	2025-10-06

Creating Views :

- Create a view for the final data and import to gold layer
- --> CREATE VIEW gold.dim_customers AS

```
CREATE VIEW gold.dim_customers AS
```

```
SELECT
```

```
    ROW_NUMBER() OVER (ORDER BY cst_id) AS customer_key,
    ci.cst_id AS customer_id,
    ci.cst_key AS customer_number,
    ci.cst_firstname AS first_name,
    ci.cst_lastname AS last_name,
    la.centry AS country,
    ci.cst_marital_status AS marital_status,
    CASE WHEN ci.cst_gndr != 'n/a' THEN ci.cst_gndr -- CRM is the Master for Gender Info
    ELSE COALESCE(ca.gen, 'n/a')
    END AS gender,
    ca.bdate AS birthdate,
    ci.cst_create_date AS create_date
FROM silver.crm_cust_info ci -- ci --> is alias for the table
LEFT JOIN silver.erp_cust_az12 ca
ON      ci.cst_key = ca.cid
LEFT JOIN silver.erp_loc_a101 la
ON      ci.cst_key = la.cid
```

100 %

Results Messages

	customer_key	customer_id	customer_number	first_name	last_name	country	marital_status	gender	birthdate	create_date
1	1	11000	AW00011000	Jon	Yang	Australia	Married	Male	1971-10-06	2025-10-06
2	2	11001	AW00011001	Eugene	Huang	Australia	Single	Male	1976-05-10	2025-10-06
3	3	11002	AW00011002	Ruben	Torres	Australia	Married	Male	1971-02-09	2025-10-06
4	4	11003	AW00011003	Christy	Zhu	Australia	Single	Female	1973-09-14	2025-10-06

Dimensional Table - Product :

```
SELECT
```

```
    pn.prd_id,
    pn.cat_id,
    pn.prd_key,
    pn.prd_nm,
    pn.prd_cost,
    pn.prd_line,
    pn.prd_start_dt,
    pn.prd_end_dt
FROM silver.crm_prd_info pn
```

100 %

Results Messages

	prd_id	cat_id	prd_key	prd_nm	prd_cost	prd_line	prd_start_dt	prd_end_dt
1	478	AC_BC	BC-M005	Mountain Bottle Cage	4	Mountain	2013-07-01	NULL
2	479	AC_BC	BC-R205	Road Bottle Cage	3	Road	2013-07-01	NULL
3	477	AC_BC	WB-H098	Water Bottle - 30 oz.	2	Other Sales	2013-07-01	NULL
4	483	AC_BR	RA-H123	Hitch Rack - 4-Bike	45	Other Sales	2013-07-01	NULL
5	486	AC_BS	ST-1401	All-Purpose Bike Stand	59	Mountain	2013-07-01	NULL
6	484	AC_CL	CL-9009	Bike Wash - Dissolver	3	Other Sales	2013-07-01	NULL

- Perform joins

```

SELECT
    pn.prđ_id,
    pn.cat_id,
    pn.prđ_key,
    pn.prđ_nm,
    pn.prđ_cost,
    pn.prđ_line,
    pn.prđ_start_dt,
    pc.cat,
    pc.subcat,
    pc.maintenance
FROM silver.crm_prđ_info pn
LEFT JOIN silver.erp_px_cat_glv2 pc
ON     pn.cat_id = pc.id
WHERE prđ_end_dt IS NULL -- Filters out all historical Data

```

100 %

Results Messages

	prđ_id	cat_id	prđ_key	prđ_nm	prđ_cost	prđ_line	prđ_start_dt	cat	subcat	maintenance
1	478	AC_BC	BC-M005	Mountain Bottle Cage	4	Mountain	2013-07-01	Accessories	Bottles and Cages	No
2	479	AC_BC	BC-R205	Road Bottle Cage	3	Road	2013-07-01	Accessories	Bottles and Cages	No
3	477	AC_BC	WB-H098	Water Bottle - 30 oz.	2	Other Sales	2013-07-01	Accessories	Bottles and Cages	No
4	483	AC_BR	RA-H123	Hitch Rack - 4-Bike	45	Other Sales	2013-07-01	Accessories	Bike Racks	Yes
5	486	AC_RS	ST-1401	All-Purpose Bike Stand	59	Mountain	2013-07-01	Accessories	Bike Stands	No

- Now Rename & Order the columns logically

```

SELECT
    pn.prđ_id AS product_id,
    pn.prđ_key AS prodduce_number,
    pn.prđ_nm AS product_name,
    pn.cat_id AS category_id,
    pc.cat AS category,
    pc.subcat AS subcategory,
    pc.maintenance,
    pn.prđ_cost AS cost,
    pn.prđ_line AS product_line,
    pn.prđ_start_dt AS start_date

FROM silver.crm_prđ_info pn
LEFT JOIN silver.erp_px_cat_glv2 pc
ON     pn.cat_id = pc.id
WHERE prđ_end_dt IS NULL -- Filters out all historical Data

```

100 %

Results Messages

	product_id	prodduce_number	product_name	category_id	category	subcategory	maintenance	cost	product_line	start_date
1	478	BC-M005	Mountain Bottle Cage	AC_BC	Accessories	Bottles and Cages	No	4	Mountain	2013-07-01
2	479	BC-R205	Road Bottle Cage	AC_BC	Accessories	Bottles and Cages	No	3	Road	2013-07-01
3	477	WB-H098	Water Bottle - 30 oz.	AC_BC	Accessories	Bottles and Cages	No	2	Other Sales	2013-07-01
4	483	RA-H123	Hitch Rack - 4-Bike	AC_BR	Accessories	Bike Racks	Yes	45	Other Sales	2013-07-01
5	486	ST-1401	All-Purpose Bike Stand	AC_RS	Accessories	Bike Stands	No	59	Mountain	2013-07-01

- Here again when we notice , this table gives more information about the product but not how much it is sold etc etc not transactional. Hence this is a Dimensional Table.

```
CREATE VIEW gold.dim_products AS
```

```
SELECT
```

```
    ROW_NUMBER() OVER (ORDER BY pn.prd_start_dt, pn.prd_key) AS product_key,  
    pn.prd_id AS product_id,  
    pn.prd_key AS prodduce_number,  
    pn.prd_nm AS product_name,  
    pn.cat_id AS category_id,  
    pc.cat AS category,  
    pc.subcat AS subcategory,  
    pc.maintenance,  
    pn.prd_cost AS cost,  
    pn.prd_line AS product_line,  
    pn.prd_start_dt AS start_date
```

```
FROM silver.crm_prd_info pn
```

```
LEFT JOIN silver.erp_px_cat_glv2 pc
```

```
ON      pn.cat_id = pc.id
```

```
WHERE prd_end_dt IS NULL -- Filters out all historical Data
```

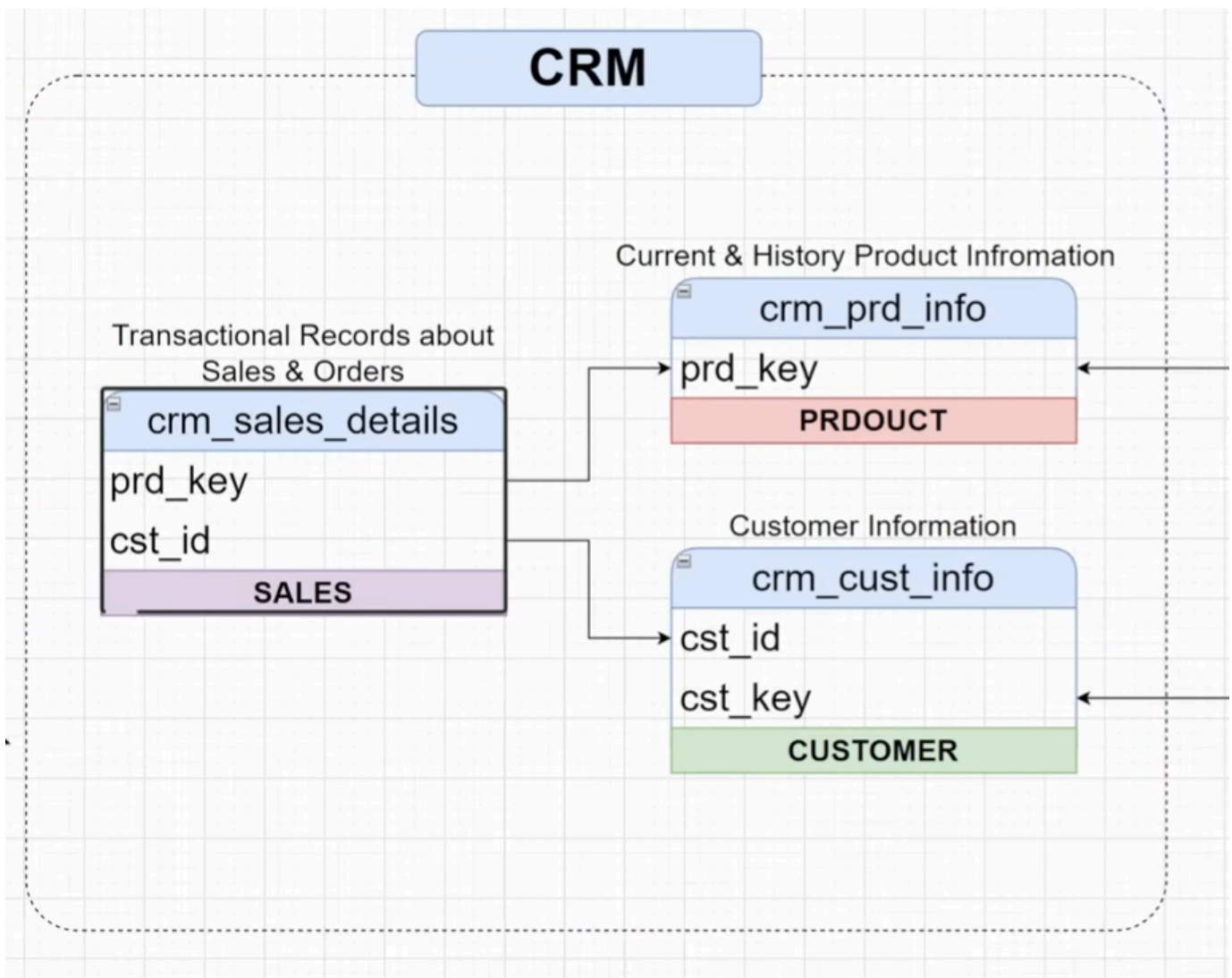
100 %

Messages

Commands completed successfully.

Completion time: 2026-03-18T19:09:58.7074520+05:30

Fact Tables :



For Transactional data we have data from CRM only here that is **crm_sales_details**,

1. First view the table as a preview


```

SELECT
sd.sls_ord_num,
sd.sls_prd_key,
sd.sls_cust_id,
sd.sls_order_dt,
sd.sls_ship_dt,
sd.sls_due_dt,
sd.sls_sales,
sd.sls_quantity,
sd.sls_price
FROM silver.crm_sales_details sd

```

100 %

Results Messages

	sls_ord_num	sls_prd_key	sls_cust_id	sls_order_dt	sls_ship_dt	sls_due_dt	sls_sales	sls_quantity	sls_price
1	SO75001	SJ-0194-X	11123	2014-01-25	2014-02-01	2014-02-06	54	1	54
2	SO75002	HL-U509-R	17305	2014-01-25	2014-02-01	2014-02-06	35	1	35
3	SO75003	LJ-0192-X	14321	2014-01-25	2014-02-01	2014-02-06	50	1	50
4	SO75004	TT-R982	11229	2014-01-25	2014-02-01	2014-02-06	4	1	4
5	SO75004	HL-U509-R	11229	2014-01-25	2014-02-01	2014-02-06	35	1	35
6	SO75005	TT-R982	19640	2014-01-25	2014-02-01	2014-02-06	4	1	4
7	SO75005	TI-R092	19640	2014-01-25	2014-02-01	2014-02-06	21	1	21
8	SO75005	PK-7098	19640	2014-01-25	2014-02-01	2014-02-06	2	1	2
9	SO75006	TI-T723	12945	2014-01-25	2014-02-01	2014-02-06	29	1	29
10	SO75006	TT-T092	12945	2014-01-25	2014-02-01	2014-02-06	5	1	5
11	SO75006	BC-R205	12945	2014-01-25	2014-02-01	2014-02-06	9	1	9
12	SO75006	WB-H098	12945	2014-01-25	2014-02-01	2014-02-06	5	1	5
13	SO75006	HY-1023-70	12945	2014-01-25	2014-02-01	2014-02-06	55	1	55

2.

FROM silver.crm_sales_details sd

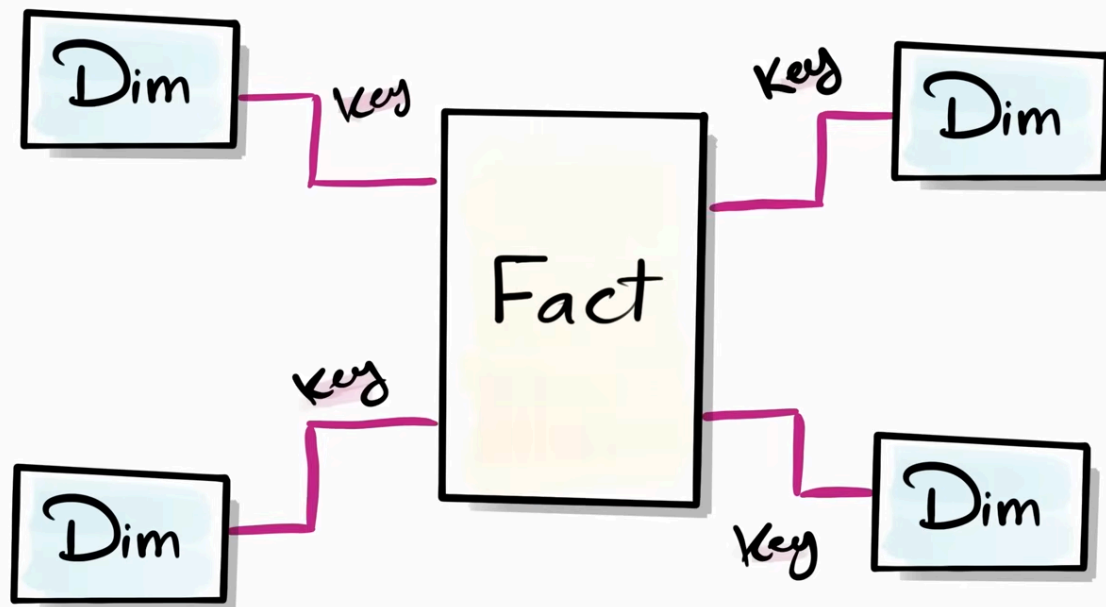
KEYS

DATES

MEASURES

sls_ord_num	sls_prd_key	sls_cust_id	sls_order_dt	sls_ship_dt	sls_due_dt	sls_sales	sls_quantity	sls_price
SO43697	BK-R93R-62	21768	2010-12-29	2011-01-05	2011-01-10	3578	1	3578
SO43698	BK-M82S-44	28389	2010-12-29	2011-01-05	2011-01-10	3400	1	3400
SO43699	BK-M82S-44	25863	2010-12-29	2011-01-05	2011-01-10	3400	1	3400
SO43700	BK-R50B-62	14501	2010-12-29	2011-01-05	2011-01-10	699	1	699
SO43701	BK-M82S-44	11003	2010-12-29	2011-01-05	2011-01-10	3400	1	3400
SO43702	BK-R93R-44	27645	2010-12-30	2011-01-06	2011-01-11	3578	1	3578
SO43703	BK-R93R-62	16624	2010-12-30	2011-01-06	2011-01-11	3578	1	3578

As we can see lot of transactional dates & event and many Numerical values / measures and has a lot of Id's



crm sales details sd

Original IDs

sls_prd_key	sls_cust_id
BK-R93R-62	2168
BK-M82S-44	28389
BK-M82S-44	25863
BK-R50B-62	14501
BK-M82S-44	11003
BK-R93R-44	27645
BK-R93R-62	16624
BK-M82B-48	11005

Since the original keys are created from the source system , its preferred to not touch them until it is really necessary , It is always better to create a **Surrogate Key**

- This also makes it easy to use the dimensions surrogate keys instead of IDs to easily connect facts with dimensions
- **This process is called data Lookup**

```

SELECT
  sd.sls_ord_num,
  pr.product_key,
  cu.customer_key,
  sd.sls_cust_id,
  sd.sls_order_dt,
  sd.sls_ship_dt,
  sd.sls_due_dt,
  sd.sls_sales,
  sd.sls_quantity,
  sd.sls_price
FROM silver.crm_sales_details sd
LEFT JOIN gold.dim_products pr
ON sd.sls_prd_key = pr.product_number
LEFT JOIN gold.dim_customers cu
ON sd.sls_cust_id = cu.customer_id

```

0 %

Results Messages

	sls_ord_num	product_key	customer_key	sls_cust_id	sls_order_dt	sls_ship_dt	sls_due_dt	sls_sales	sls_quantity	sls_price
1	SO43697	20	10769	21768	2010-12-29	2011-01-05	2011-01-10	3578	1	3578
2	SO43703	20	5625	16624	2010-12-30	2011-01-06	2011-01-11	3578	1	3578
3	SO43713	20	16602	27601	2011-01-02	2011-01-09	2011-01-14	3578	1	3578
4	SO43721	20	2591	13590	2011-01-03	2011-01-10	2011-01-15	3578	1	3578
5	SO43735	20	5523	16522	2011-01-07	2011-01-14	2011-01-19	3578	1	3578
6	SO43739	20	2564	13563	2011-01-08	2011-01-15	2011-01-20	3578	1	3578

- Rename the columns for better understandability

```

SELECT
  sd.sls_ord_num AS order_number,
  pr.product_key,
  cu.customer_key,
  sd.sls_cust_id AS customer_id,
  sd.sls_order_dt AS order_date,
  sd.sls_ship_dt AS shipping_date,
  sd.sls_due_dt AS due_date,
  sd.sls_sales AS sales_amount,
  sd.sls_quantity AS quantity,
  sd.sls_price AS price
FROM silver.crm_sales_details sd
LEFT JOIN gold.dim_products pr
ON sd.sls_prd_key = pr.product_number
LEFT JOIN gold.dim_customers cu
ON sd.sls_cust_id = cu.customer_id

```

10 %

Results Messages

	order_number	product_key	customer_key	customer_id	order_date	shipping_date	due_date	sales_amount	quantity	price
1	SO75001	278	124	11123	2014-01-25	2014-02-01	2014-02-06	54	1	54
2	SO75002	246	6306	17305	2014-01-25	2014-02-01	2014-02-06	35	1	35
3	SO75003	251	3322	14321	2014-01-25	2014-02-01	2014-02-06	50	1	50
4	SO75004	290	230	11229	2014-01-25	2014-02-01	2014-02-06	4	1	4
5	SO75004	246	230	11229	2014-01-25	2014-02-01	2014-02-06	35	1	35

- We have a schema / Order to sort the columns into logical groups in Fact Table
 - Dimension Keys
 - Dates
 - Measures

DIMENSION KEYS			DATES			MEASURES		
order_number	product_key	customer_key	order_date	shipping_date	due_date	sales_amount	quantity	price
SO43697	20	10769	2010-12-29	2011-01-05	2011-01-10	3578	1	3578
SO43698	9	17390	2010-12-29	2011-01-05	2011-01-10	3400	1	3400
SO43699	9	14864	2010-12-29	2011-01-05	2011-01-10	3400	1	3400
SO43700	41	3502	2010-12-29	2011-01-05	2011-01-10	699	1	699
SO43701	9	4	2010-12-29	2011-01-05	2011-01-10	3400	1	3400
SO43702	16	16646	2010-12-30	2011-01-06	2011-01-11	3578	1	3578

Create the View for Gold Fact Table

```
CREATE VIEW gold.fact_sales AS
SELECT
  sd.sls_ord_num AS order_number,
  pr.product_key,
  cu.customer_key,
  sd.sls_cust_id AS customer_id,
  sd.sls_order_dt AS order_date,
  sd.sls_ship_dt AS shipping_date,
  sd.sls_due_dt AS due_date,
  sd.sls_sales AS sales_amount,
  sd.sls_quantity AS quantity,
  sd.sls_price AS price
FROM silver.crm.sales_details sd
```

- As a quality check , Try to join the Fact Table with dimensional tables using the keys

```
SELECT *
FROM gold.fact_sales f -- Fact Table
LEFT JOIN gold.dim_customers c -- Dimensional Table
ON c.customer_key = f.customer_key
WHERE c.customer_key IS NULL
```

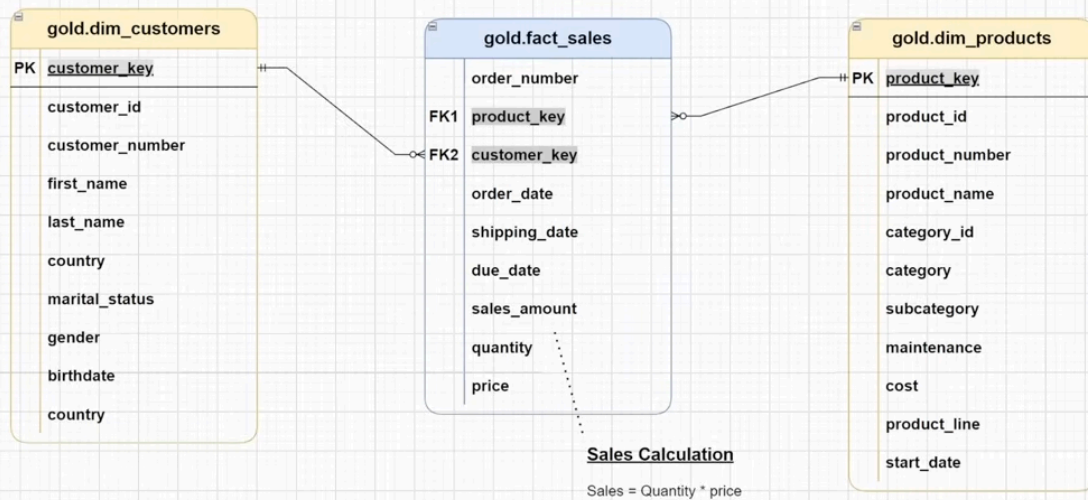
100 %

Results Messages

order_number	product_key	customer_key	customer_id	order_date	shipping_date	du
--------------	-------------	--------------	-------------	------------	---------------	----

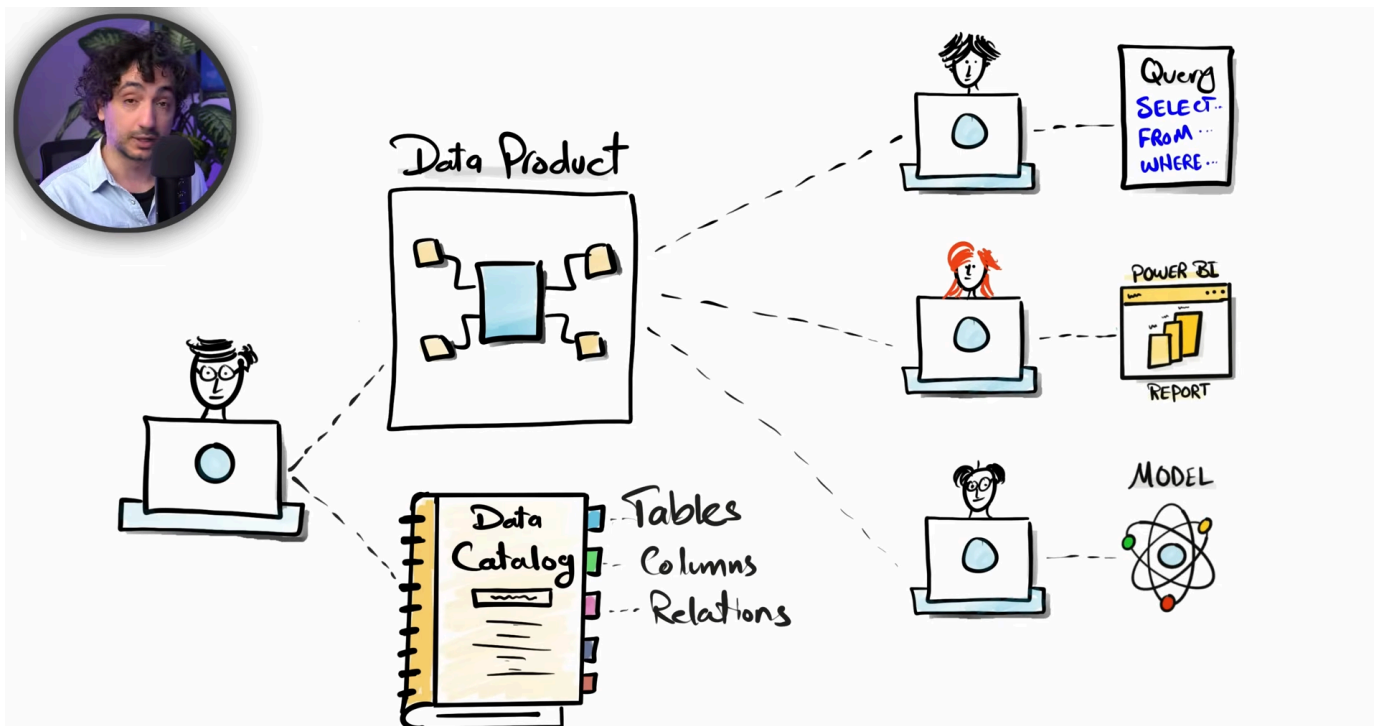
- Final Data Model : (Star Schema)

Sales Data Mart (Star Schema)



Building Data Catalog :

- A Data Catalog , is a index of changes and reference which provides an idea and framework of what we did to anyone who is next going to do any changes and upgradation



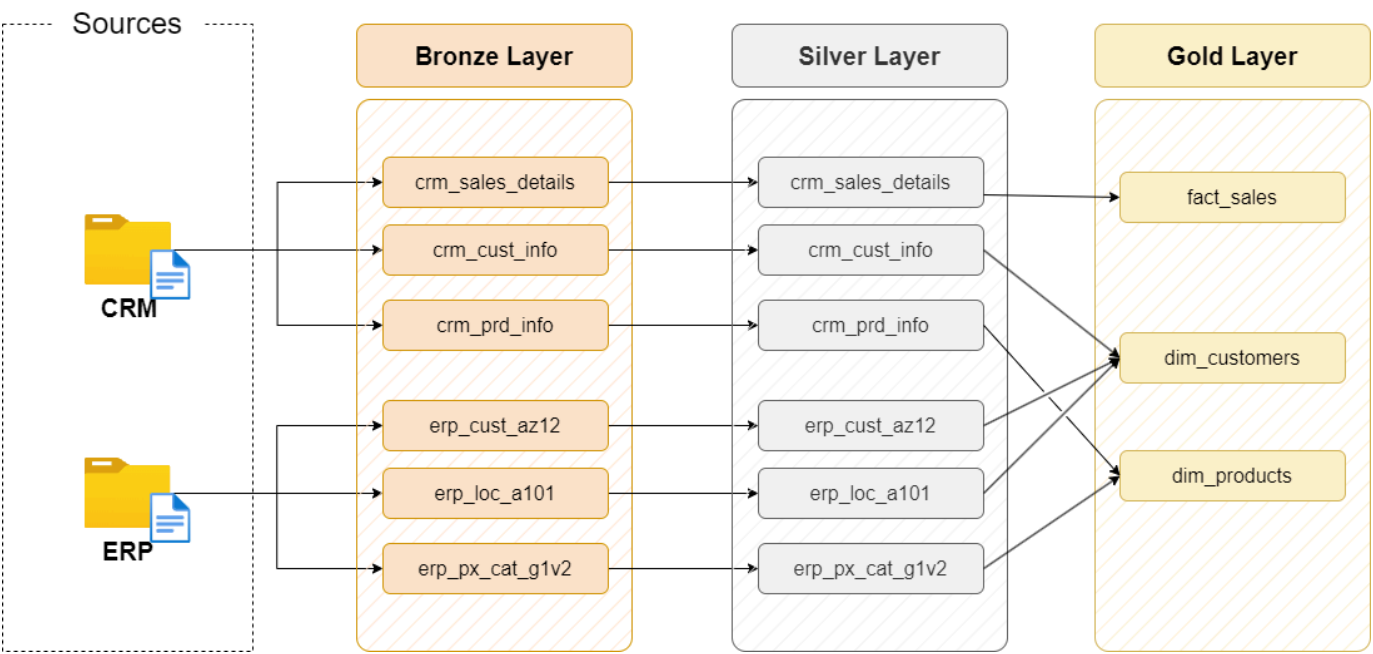
- Describe Each Column and table,
 - Why is it used
 - Column Name , Datatype , Description

• Changes

- **Purpose:** Provides information about the products and their attributes.
- **Columns:**

Column Name	Data Type	Description
product_key	INT	Surrogate key uniquely identifying each product record in the product dimension table.
product_id	INT	A unique identifier assigned to the product for internal tracking and referencing.
product_number	NVARCHAR(50)	A structured alphanumeric code representing the product, often used for categorization or inventory.
product_name	NVARCHAR(50)	Descriptive name of the product, including key details such as type, color, and size.
category_id	NVARCHAR(50)	A unique identifier for the product's category, linking to its high-level classification.
category_name	NVARCHAR(50)	The broader classification of the product (e.g., Bikes, Computers) to group related items.

Final Data Flow Diagram :



Conclusion :

This project helped me gain hands-on experience in designing and implementing a modern data warehouse using Medallion Architecture (Bronze, Silver, and Gold layers). I learned how to build ETL pipelines, perform data cleansing and transformation, and model data using fact and dimension tables in a star schema. Additionally, I developed SQL-based analytics and reporting to generate actionable insights. Through this project, I strengthened my skills in data engineering, data modeling, and analytics, and successfully upskilled myself by completing an end-to-end data warehouse and analytics solution.